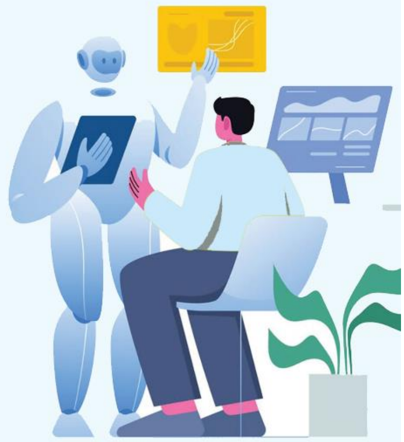
VOCACARE

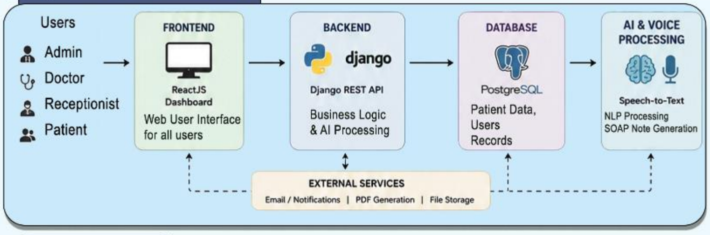
AI POWERED HOSPITAL INFORMATION MANAGEMENT SYSTEM

Key Features

- AI-Assisted Documentation
- Appointment & Queue Management
- Patient Records Management
- Secure Communication
- Billing & Reports
- Role-Based Access



System Architecture



Use Cases

AI-Assisted Medical Documentation
Doctor records consultation - AI transcribes and extracts information - Generate SOAP Notes - Doctor verifies and saves

Appointment & Queue Management
Patient books appointment System assigns queue Real-time queue updates and status tracking

Patient Record Access
Authorized users access patient history, prescriptions, reports, and visit details securely.

Prescription Management
Doctor creates prescription - Stored and accessible digitally Patients can view and download

Secure Communication & Notifications
System and users exchange messages - Appointment, billing, and report notifications via dashboard/email

Administrative Dashboard
Manage users, monitor activities, view analytics, handle billing, and system operations

Tech Stack

Backend
Django, Django REST Framework

Frontend
ReactJS, Tailwind CSS

Database
PostgreSQL

AI & Voice Processing
Speech Recognition, NLP (Voice-to-Text, SOAP Notes)

Reporting
PDF Generation and Cloudinary Storage

Communication
Email APIs, In-System Messaging

Authentication
Django Auth, Role-Based Access

APIs
REST APIs, JSON

Version Control
Git, GitHub

SUPERVISOR
Mr Aamir Zia
Department of Computer Science
University of Sargodha

GROUP MEMBERS:
Ali Akbar - BSCS51F22R036
Najee Ullah Noon - BSCS51F22S043

BATCH: **BSCS SELF SS1 (2022-2026)**
Department of Computer Science
University of Sargodha

"AI Powered Hospital Information Management System"

Project Code: HIMS-AI-2526

Project Team:

Ali Akbar BSCS51F22R036

Muhammad Najee Ullah Noon BSCS51F22S043

Project Supervisor:

Mr. Aamir Zia

Project Manager:

Dr. Muhammad Ilyas

Submission Date:

21-05-2026

DEPARTMENT OF CS & IT

UNIVERSITY OF SARGODHA, SARGODHA

SESSION (2022-2026)

Table of Contents

Chapter 01: Introduction

1.1 Purpose of the document

This document has been prepared to provide a comprehensive overview of the design, development, and implementation of the AI Powered Hospital Information Management System (HIMS), a smart healthcare solution aimed at improving hospital operations and enhancing patient care through intelligent automation. The purpose of this document is not only to explain the technical structure of the system but also to describe the planning, methodologies, and problem-solving approaches used during its development.

It serves as a reference for all stakeholders involved in the project, including the development team, hospital administrators, healthcare professionals, and academic evaluators. By clearly presenting the project objectives, system architecture, features, workflows, user roles, implementation strategies, and testing approaches, the document ensures that the system is understood from both technical and operational perspectives.

Additionally, the document helps maintain transparency throughout the development lifecycle, supports effective decision-making, and provides a foundation for future maintenance, scalability, and feature enhancements. Whether the system is being reviewed for academic evaluation, practical deployment, or future expansion within healthcare institutions, this document provides all essential information regarding the proposed solution and its functionalities.

1.2 Project Overview

Modern hospitals often struggle with inefficiencies caused by manual data entry, fragmented record-keeping, and time-consuming administrative processes. These challenges lead to delays in patient care, errors in documentation, and an increased workload on medical staff.

To address these issues, the AI-Powered Hospital Information Management System (HIMS) is proposed as an intelligent, automated platform that integrates speech recognition and natural language processing to streamline hospital workflows.

The system functions as follows:

1. **AI-Based Conversation Recording and Analysis:** The system listens to doctor–patient interactions in real time, identifies the speaker, and extracts key medical information such as symptoms, diagnosis, prescribed treatments, and follow-up details.
2. **Doctor Confirmation and Data Storage:** Before saving, the extracted data is displayed to the doctor for verification. Once confirmed, it is automatically stored in a centralized hospital database.
3. **Dedicated Dashboards:**

- **Doctor Dashboard:** Enables doctors to view patient histories, review notes, and manage consultations.
- **Admin Dashboard:** Allows hospital administrators to manage records, monitor data flow, and ensure system compliance.

- **Patient Portal:** Provides patients with access to their medical history, prescriptions, and reports.

4. **Appointment Management**: Real-time queue tracking and appointment management help minimize overcrowding and waiting time in hospitals.
5. **Medical History**: Digital record management ensures easy and secure access to patient history, prescriptions, and billing information.

The core objective of this project is to reduce administrative workload, improve record accuracy, and enable real-time access to patient information. By leveraging advanced technologies such as deepgram nova 3 for speech-to-text processing, GPT-based models (Open AI GPT4.1) for data extraction, and PostgreSQL for secure data management, the proposed system enhances hospital efficiency and promotes the adoption of AI-driven healthcare solutions.

1.3 Project Scope

The scope of the AI-powered Hospital Information Management System (HIMS) is defined by its focus on improving patient record management, reducing administrative workload, and supporting accurate medical documentation through AI assistance, while keeping doctors in control of validation. The project is specifically designed for healthcare institutions and has clear boundaries on what it will and will not cover.

Inclusions (What the System Will Do):

- **AI-Assisted Medical Note Creation:** "The system captures doctor-patient conversations, highlights relevant medical details (symptoms, diagnoses, tests), and presents them for doctor confirmation before saving to the database [4]."
- **Smart Appointment and Queue Management:** Patients can book appointments on a first-come-first-serve basis and monitor their queue position in real-time to minimize hospital waiting time and overcrowding.
- **Controlled Patient Self-Registration:** Patients can create their own accounts through the registration system; however, the accounts will remain inactive until verified and approved by authorized hospital staff to ensure security and authenticity of patient information.
- **Centralized Patient Record Management:** Hospitals can maintain structured medical histories, accessible to authorized staff across visits.
- **Prescription Management System:** The platform stores doctor-specific prescription history, allowing healthcare providers to review previous medications and treatment records for continuity of care.
- **Integrated Staff Dashboard:** Enables staff to verify or edit AI-suggested notes, manage patient data, and oversee hospital operations such as scheduling and reporting.
- **Integrated Communication Module:** The system provides a secure messaging platform for healthcare-related communication, follow-ups, notifications, and information exchange within the hospital management environment.
- **Secure Patient Access:** Patients can receive their reports digitally through their profile or via staff-provided printed documents.
- **Transparent Billing and Report Access:** Patients can access billing information, payment details, and downloadable medical reports digitally through their accounts.

- **Healthcare Data Analysis:** Aggregated hospital data can be used to analyze patient trends, workload distribution, consultation frequency, and operational efficiency.

Exclusions (What the System Will Not Do):

- **Autonomous Medical Decision-Making:** The system will not independently diagnose diseases, prescribe medicines, or recommend treatments without doctor involvement and approval.
- **Cross-Hospital Data Sharing:** Patient data will not automatically transfer between hospitals unless formally integrated with external systems.
- **Non-Medical Hospital Operations:** The system will not handle unrelated tasks such as payroll, pharmacy stock management, or facility maintenance.
- **Emergency Response Management:** The platform is not intended to function as an emergency dispatch or ambulance coordination system.
- **Advanced AI Disease Prediction:** The current version of the system will not include predictive analytics for disease forecasting or automated medical risk assessment.
- **Multilingual Limitation:** The system currently supports English and Urdu only and does not provide support for additional regional or international languages in the present implementation.

1.4 Project Timeline

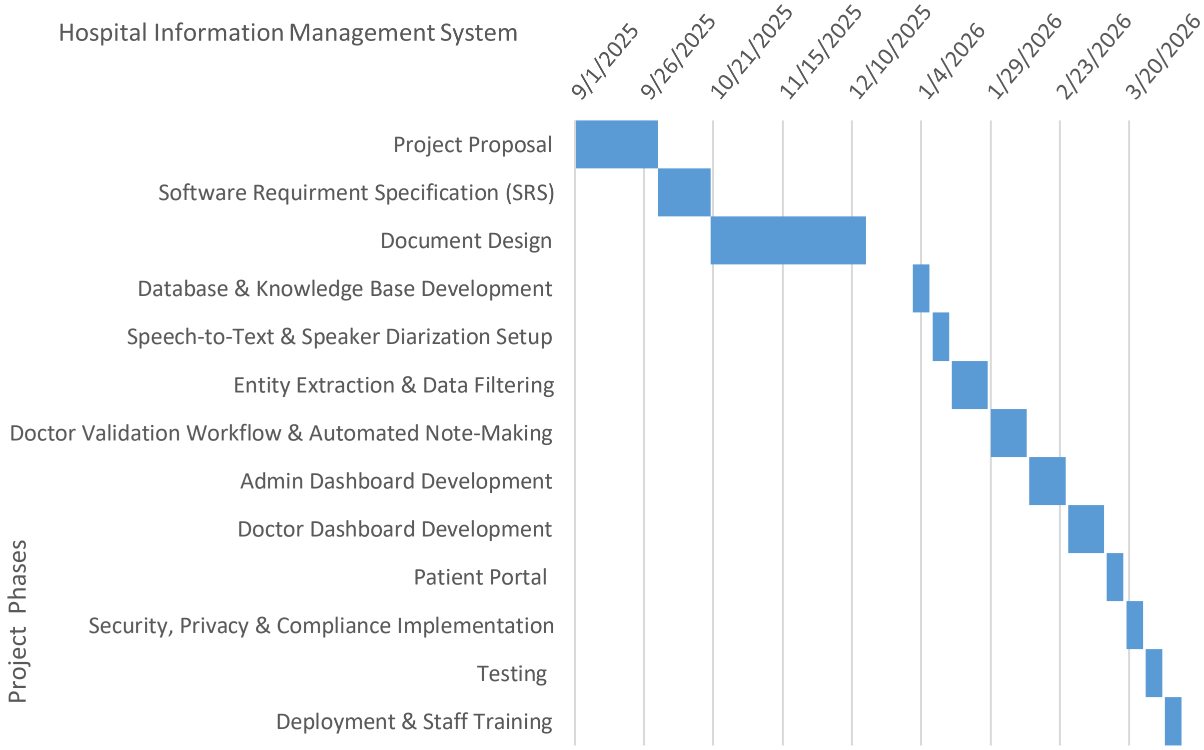


Figure 1: Project Timeline - Gantt chart

Chapter 02: Overall System Description

The **AI-Powered Hospital Information Management System (HIMS)** is designed for multiple classes of users within a healthcare setting, each with specific roles, technical familiarity, and system access levels.

2.1 User Characteristics

2.1.1 Primary Users – Doctors

Doctors are the main users of the system, responsible for patient diagnosis, treatment, and record validation. They interact with the system during consultations, where the AI automatically records and transcribes doctor-patient conversations, extracts relevant medical information (such as symptoms, diagnoses, and prescribed medications), and presents it for confirmation. Doctors then review, edit if necessary, and approve the extracted information before it is stored in the database.

This user group requires an interface that is quick, accurate, and minimally intrusive to their workflow. Since doctors are typically focused on patient care rather than data entry, the system focuses on automation, clarity, and reliability in displaying AI-generated medical summaries.

2.1.2 Secondary Users – Administrative Staff

Administrative staff, including hospital managers and data officers, form the second key user group. Their role involves managing hospital data and maintaining compliance with privacy regulations, and monitoring daily operations. They interact with the admin dashboard to view patient activity logs, manage user accounts, oversee system performance, and generate analytical reports for hospital management.

These users are moderately technical and require secure login credentials. The dashboard is designed with a focus on usability and efficient data management, allowing them to perform tasks without deep technical expertise.

2.1.3 Operational Users – Receptionists

Receptionists are operational users responsible for managing front-desk activities and assisting doctors in daily patient handling. Each receptionist is assigned to a specific doctor and can only manage operations related to that doctor. Their responsibilities include patient registration, appointment scheduling, queue management, verification of patient details, and assisting patients during hospital visits.

Receptionists interact with a restricted dashboard that provides access only to the assigned doctor’s appointments, patient queues, and related operational data. This role-based access control helps maintain patient privacy and ensures organized workflow management within the hospital.

The system interface for receptionists is designed to be simple, responsive, and easy to use, enabling efficient handling of routine administrative tasks with minimal technical complexity.

2.1.4 Tertiary Users – Patients

Patients are indirect users of the system who benefit from AI-generated and doctor-verified medical records. They can access their visit history, prescriptions, and reports through a patient portal or via hospital assistance. Since some patients may lack technological access or literacy, the system ensures that staff can retrieve and provide printed or digital copies of medical information when needed.

This group's interaction is limited but essential for ensuring transparency, continuous care, and patient empowerment in the healthcare process.

2.2 Operating environment

The **AI-Powered Hospital Information Management System (HIMS-AI)** will operate within a secure, cloud-enabled healthcare environment designed to ensure efficiency, scalability, and data protection. The following outlines the key components of its operating context:

1. **Web-Based Accessibility:**
   The HIMS platform is entirely web-based, allowing access through standard browsers such as Google Chrome, Microsoft Edge, and Mozilla Firefox. Doctors, administrators, and patients can use the system from desktop or mobile devices, enabling flexibility in hospital operations.
2. **Cloud Hosting and Storage:**
   The system will be hosted on a secure cloud infrastructure to ensure high availability, automated backups, and scalability. This environment supports continuous uptime for critical hospital operations and facilitates secure remote access when needed.
3. **Hardware and Software Requirements:**
   The hospital environment will utilize standard computer systems with at least 8GB RAM and a stable internet connection. The system backend will be developed using **Django**, while the frontend will employ **React.js** for responsive interfaces. The AI models for transcription and entity extraction will be integrated through Open AI and deepgram.
4. **Database and API Integration:**
   The system will use **PostgreSQL** for storing structured medical data and patient histories. It will also integrate APIs for AI transcription, natural language processing, and authentication services to ensure smooth communication between modules.
5. **Operating System and Compatibility:**
   HIMS will be compatible with major operating systems, including **Windows**, **macOS**, and **Linux**, ensuring seamless deployment across diverse hospital IT infrastructures. The platform is also designed to integrate with existing hospital devices and systems while maintaining compliance with data privacy standards such as HIPAA.

2.3 System constraints

Despite being designed for efficiency, automation, and scalability, the **AI-Powered Hospital Information Management System (HIMS-AI)** operates under several constraints that may influence its overall performance and implementation:

- **Software Constraints:**
  The system's AI modules, including **Deepgram** for transcription and **Open AI GPT 4.1** for medical entity extraction, depend on stable third-party APIs and pre-trained models. Any updates or service interruptions in these tools could affect system accuracy and performance.
- **Hardware Constraints:**
  The transcription and AI processing components require systems with sufficient computing power (minimum 8GB RAM and multi-core processors). Hospitals with outdated systems or limited network infrastructure may experience slower processing times.
- **Cultural and Language Constraints:**
  Since doctor-patient conversations may involve a mix of **Urdu and English**, the system's multilingual transcription model may face limitations in perfectly recognizing accents, dialects, or regional variations, possibly affecting the data accuracy.
- **Legal and Privacy Constraints:**
  The system must comply with healthcare data protection laws (e.g., HIPAA or local equivalents). Strict consent is required for recording conversations, and data encryption is mandatory to prevent misuse of sensitive patient information.
- **Environmental Constraints:**
  In busy or noisy hospital environment, background sounds can affect audio clarity and reduce transcription accuracy. Use of high-quality microphones and sound-isolated consultation rooms is recommended to mitigate this issue.
- **User Constraints:**
  Some users, such as elderly patients or non-technical staff, may struggle to interact with the web portal or dashboards. Simplified user interfaces and staff assistance will be necessary for accessibility.
- **Third-Party Component Limitations:**
  Off-the-shelf AI services (e.g., Deepgram, OpenAI GPT 4.1) may have rate limits, language support restrictions, or licensing terms that constrain large-scale deployment without enterprise subscriptions.

Chapter 03: Functional Requirements

Patient Registration and Management	• The system allows hospital staff to register new patients by entering demographic details, contact information, and medical history. Patient can register his account but it will be inactive unless verified by the system. Patients can be assigned a unique hospital ID for future tracking and appointments. • The system enables editing, updating, and searching of patient records quickly and securely.
Appointment Scheduling	• Doctors and receptionists can schedule, reschedule, or cancel patient appointments through the system. Receptionist will only assist to a specific doctor. • Patients can book appointments on a first-come-first-serve basis and monitor their queue position in real-time to minimize hospital waiting time and overcrowding. • The system ensures no time conflicts in doctor scheduling and maintains daily appointment logs.
Electronic Medical Records (EMR)	• The system stores and manages patient medical records, including diagnoses, prescriptions, and treatment history. Authorized doctors can update medical records after each visit. • Records are accessible only to authorized personnel to ensure confidentiality.

Billing and Payment Management	• The system automatically generates bills based on the services provided. • Supports multiple payment modes such as cash or online billing methods like JazzCash or Easypaisa. Payment can be refunded in case of the appointment is cancelled. • Generates invoices and maintains transaction history for audit purposes.
Doctor and Staff Management	• Allows hospital administrators to add, update, or remove doctor and staff profiles. • Stores details such as specialization, availability.
Authentication and Role Management	• Provides secure login for different user roles such as admin, doctor, nurse, pharmacist, and receptionist. • Each role has access to specific modules and data based on authorization level. • Password recovery option is available.
Data Analytics and Reporting	• The system generates statistical and analytical reports for hospital management, such as patient inflow, revenue trends, and medicine consumption. • Graphical dashboards provide insights into operational efficiency and performance. • Reports can be exported in PDF format.

Notification and Alert System	- Sends appointment reminders and bill payment alerts notifications to patients via dashboard. - Notifies staff about unprocessed bills. - Allows admins to configure notification preferences.
Backup and Recovery	- The system automatically backs up critical data on a scheduled basis to prevent loss due to system failure. - Admins can restore data from backup when necessary. - Ensures data integrity and business continuity.
Exclusions	• The system does not provide telemedicine or video consultation features. • External insurance claim processing systems are not integrated in the current version. • Real-time integration with third-party pharmacy or lab systems is outside the current project scope.

Actors	• **Patients:** Register, book appointments, and access their records. • **Doctors:** Manage appointments, update patient medical records, and view reports. • **Receptionists:** Register patients, manage appointments, and handle billing. • **Admins:** Manage staff accounts, view system analytics, and oversee hospital operations.

Chapter 04: Non-Functional Requirements

4.1 Performance Requirements

The Hospital Information Management System (HIMS) must deliver consistent, reliable, and efficient performance to ensure seamless hospital operations and patient data handling. Key performance requirements include:

- The system should retrieve patient records from the database within **20 seconds** of a query being made.
- Appointment scheduling, billing, and patient registration processes should complete within **5 seconds** of data entry.
- The system must support **up to 100 concurrent users** (doctors, nurses, and administrative staff) without performance degradation.
- Data synchronization between modules (e.g., patient registration, billing) should occur in **real-time** or within a **maximum delay of 7 seconds**.
- Report generation (e.g., discharge summaries, billing statements) should be completed within **7 seconds** for up to **5,000 records**.
- The web-based interface must load within **6 seconds** on a standard hospital network connection.

4.2 Safety Requirements

Since the HIMS operates in a healthcare environment where data accuracy and safety are critical, the system must include safeguards to prevent harm, data loss, or operational disruption. The safety requirements are as follows:

- The system should prevent unauthorized modification or deletion of critical medical or patient data.
- Automatic **data backup** must occur daily to secure patient and hospital records against accidental loss or system failure.
- In the event of a **power outage or server crash**, the system should automatically recover to the last stable state without losing transaction data.
- The system should display clear **warning messages** for invalid operations (e.g., missing patient ID, incomplete prescription entry, or failed database connection).
- Sensitive medical actions such as prescription updates or patient discharge approvals should require **authorized confirmation** before execution.
- All patient interactions and system alerts must avoid causing confusion or operational risks to staff in critical care areas.

4.3 Security Requirements

Given that the HIMS handles sensitive patient and hospital information, strong security and privacy measures are mandatory to protect data integrity and confidentiality. The system's security requirements include:

- Only **authorized users** (e.g., doctors, nurses, receptionist, and administrators) can access specific modules based on **role-based access control (RBAC)**.
- User **authentication** (via username and password) must be required for all system access points.
- All patient data—including medical history, CNIC, and billing details—must be **encrypted** both at rest and during transmission (using SSL/TLS).
- The system must maintain **audit logs** of all user activities, including record creation, modification, and deletion, for accountability and traceability.
- Session timeouts must automatically log out inactive users to prevent unauthorized access.
- Data privacy must comply with **relevant healthcare data protection policies** (such as HIPAA-equivalent or local data privacy laws).
- The system must restrict **external API access** to authorized services only, ensuring that no unauthorized third-party system can retrieve hospital data.
- Regular **security audits and vulnerability checks** should be performed to identify and mitigate potential threats.

4.4 User Documentation

To ensure ease of use, smooth adoption, and minimal training requirements, the Hospital Information Management System (HIMS) will include comprehensive documentation and in-system guidance for all user roles. The following documentation resources will be provided:

- **User Manual:**
  A detailed user manual will be provided for doctors, receptionist, and administrative staff. It will explain how to log in, manage patient records, schedule appointments and process billing through step-by-step instructions and annotated screenshots.
- **Admin Guide:**
  A separate administrative guide will outline procedures for managing user accounts, configuring security settings, performing data backups, and maintaining system performance.
- **Troubleshooting Guide:**
  This document will help users diagnose and resolve common issues—such as login failures, network connectivity problems, or missing data entries—without requiring technical support. Each issue will include cause explanations and recommended actions.
- **Built-in Help Documentation:**
  The system interface will include a “Help” or “Support” section that offers quick access to FAQs, tooltips, and contextual help directly within the dashboard and form screens.
- **Training and Tutorial Modules:**
  Interactive, step-by-step tutorials will guide hospital staff on essential tasks, such as recording patient information, generating reports, and updating treatment details. Short video guides and practice simulations may also be included for enhanced understanding.
- **Version Control and Updates:**
  All documentation will be version-controlled and updated regularly to reflect system enhancements, security patches, and new feature rollouts.

Chapter 05: Assumptions and Dependencies

5.1 Assumptions

- The hospital's medical staff and administrative personnel will cooperate fully in adopting the new HIMS platform and provide accurate input data during initial setup and daily usage.
- The hospital's existing hardware infrastructure (computers, microphones, and secure servers) will meet the minimum system requirements for smooth operation.
- Internet connectivity within the hospital premises will remain stable to enable real-time data synchronization and AI model processing.
- Doctor-patient conversation recordings will be conducted in environments with minimal background noise to ensure the accuracy of the speech-to-text and entity extraction modules.
- All users—including doctors, nurses, and administrative staff—will receive basic training on how to use the system effectively.
- The hospital's policies and workflows will remain consistent during the system's development and deployment phases, ensuring alignment between technical and operational procedures.

5.2 Dependencies

- The AI modules depend on external speech-to-text and natural language processing APIs for transcribing and extracting key medical data from doctor-patient conversations.
- The system relies on a stable connection to the hospital's central database for data storage and retrieval of patient records, diagnoses, and treatment histories.
- Regular data updates and maintenance must be performed by authorized IT personnel to ensure accuracy and prevent inconsistencies.
- Integration with existing hospital modules depends on the availability and compatibility of their APIs or export functionalities.
- The system's deployment and maintenance depend on compliance with healthcare regulations such as HIPAA or local data protection laws to ensure security and confidentiality.
- Timely software updates, API key renewals, and support from external AI service providers are necessary to maintain full functionality and performance.

Chapter 06: Use Cases

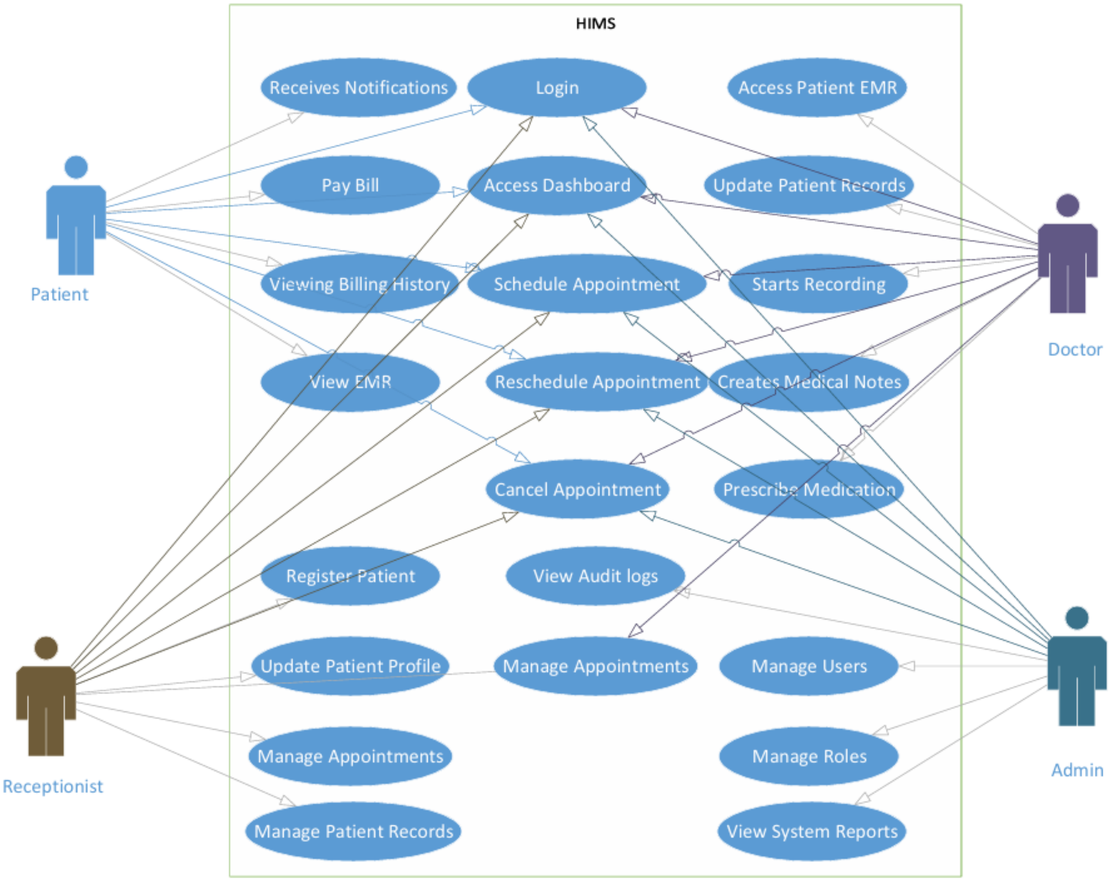


Figure 6.1 Use Case Diagram

Use Case 1: AI-Assisted Medical Documentation

Actors: Doctors

Use case ID: 21

Pre-condition:
The doctor must be authenticated into the system, and the voice recording service must be active.

Scenarios:

Step#	Action	Software Reaction
1	Doctor starts consultation session.	The system activates voice recording and prepares the consultation workspace.
2	Doctor and patient communicate during consultation.	The system captures and transcribes the conversation in real time.

3	Doctor finishes consultation.	The AI extracts medical details such as symptoms, diagnosis, medications, and recommendations.
4	Doctor reviews generated notes.	The system displays structured medical reports and SOAP notes for verification.
5	Doctor confirms or edits the notes.	The verified consultation report is securely stored in the database.

Alternate Scenarios:

- If voice recognition fails or audio quality is poor, the doctor can manually edit or enter medical notes.
- If the AI generates incomplete information, the system allows manual correction before saving.

Post Conditions:

Step	Description
1	Verified consultation records are stored securely in the database.
2	Patient medical history is updated with the latest consultation data.

Use Case 2: Appointment and Queue Management

Actors: Patients, Receptionists

Use case ID: 22

Pre-condition:
Patient account must exist and the doctor schedule must be available in the system.

Scenarios

Step#	Action	Software Reaction
1	Patient requests an appointment.	The system checks doctor availability and assigns an appointment slot.
2	Receptionist confirms appointment details.	The system generates a queue position for the patient.
3	Patient checks queue status.	The system displays real-time queue updates and estimated waiting information.

4	Patient arrives at hospital.	The system updates queue progression and consultation status.

Alternate Scenarios:

- If appointment slots are full, the system suggests the next available schedule.
- If the patient misses the appointment, the system marks the session as missed or rescheduled.

Post Conditions:

Step	Description
1	Appointment and queue data are stored in the system.
2	Patients receive updated scheduling and queue information.

Use Case 3: Patient Record Access

Actors: Doctors, Patients, Receptionists

Use case ID: 23

Pre-condition:
Authorized users must be logged into the system.

Scenarios

Step#	Action	Software Reaction
1	User searches for a patient record.	The system retrieves the patient’s medical history from the database.
2	Doctor reviews previous consultations and prescriptions.	The system displays reports, prescriptions, and visit history.
3	Patient requests downloadable reports.	The system generates downloadable PDF reports.
4	Receptionist verifies patient details.	The system updates or confirms patient information if required.

Alternate Scenarios:

- If the requested record does not exist, the system displays an appropriate notification.
- If unauthorized access is attempted, the system blocks access and logs the activity.

Post Conditions

Step	Description

1	Authorized users access accurate patient information securely.
2	Medical records remain updated and centrally managed.

Use Case 4: Prescription Management

Actors: Doctors, Patients

Use case ID: 24

Pre-condition:
Patient consultation must be completed and verified by the doctor.

Scenarios

Step#	Action	Software Reaction
1	Doctor creates or updates prescription.	The system stores prescription details in the database.
2	Doctor reviews previous medication history.	The system displays earlier prescriptions and treatment records.
3	Patient accesses prescription information.	The system provides digital prescription access and download options.

Alternate Scenarios:

- If prescription information is incomplete, the system prompts the doctor for corrections.
- If duplicate medications are detected, the system alerts the doctor for review.

Post Conditions

Step	Description
1	Prescription records are securely maintained.
2	Patients receive verified prescription information digitally.

Use Case 5: Secure Communication and Notifications

Actors: Doctors, Patients, Receptionists

Use case ID: 25

Pre-condition:
Users must be authenticated and connected to the system.

Scenarios

Step#	Action	Software Reaction

1	User sends a healthcare-related message.	The system securely delivers the message to the intended recipient.
2	System generates appointment or billing notification.	Notification is sent through dashboard alerts or email service.
3	User opens communication history.	The system displays previous messages and notifications.

Alternate Scenarios:

- If message delivery fails, the system retries or notifies the sender.
- If invalid recipient information is entered, the system displays an error message.

Post Conditions

Step	Description
1	Healthcare communication is securely maintained.
2	Users receive important alerts and updates on time.

Chapter 07: Implementation Overview

7.1 Technology Stack

Languages & Frameworks

- Python
- JavaScript
- Django
- ReactJS

Frontend

- ReactJS Web Dashboard
- HTML5, CSS3, and Tailwind CSS for responsive user interface design

Backend

- Django (Python) for backend development and business logic
- Django REST Framework (DRF) for API development and communication

Database & Storage

- PostgreSQL for structured hospital and patient data storage
- Local/Cloud storage for reports, prescriptions, and uploaded documents

Artificial Intelligence & Voice Processing

- Speech Recognition / ASR Engine for voice-to-text conversion
- NLP-based processing for extracting medical information and generating structured notes
- AI-assisted medical report and SOAP note generation

Authentication & Security

- Django Authentication System
- Role-Based Access Control (RBAC) for Admins, Doctors, Receptionists, and Patients
- Cookie-based authentication mechanisms

APIs & Data Exchange

- REST APIs for frontend-backend communication
- JSON format for data transfer
- SMTP/Email APIs for notifications and appointment alerts

Reporting & Document Generation

- PDF generation libraries for downloadable reports and prescriptions
- CSV export support for hospital records and analytics

Deployment & Hosting

- Windows environment for development
- Cloud or local server deployment support

Monitoring & Maintenance

- System activity logging
- Error monitoring and debugging tools
- Database backup and recovery support

Version Control & Collaboration

- Git
- GitHub for source code management and collaboration

7.2 System Architecture Overview

The AI Powered Hospital Information Management System (HIMS) follows a modular client-server architecture where the frontend, backend, database, and AI processing components communicate through REST APIs and secure database connections. The architecture is designed to support scalability, maintainability, and efficient healthcare workflow management.

- The **ReactJS** frontend provides dashboards and interfaces for doctors, administrators, receptionists, and patients. It communicates with the backend services through REST APIs to manage appointments, patient records, prescriptions, billing, and hospital operations.
- The **Django** backend handles business logic, authentication, role-based access control, API processing, and communication with the database. It also manages AI-assisted medical documentation and system operations.
- The **AI and voice-processing** modules capture doctor-patient conversations, process speech-to-text transcription, and generate structured medical reports and SOAP notes for doctor verification before storage.
- **PostgreSQL** is used as the central database system for storing patient records, appointment details, prescriptions, billing information, user accounts, and hospital activity logs.
- **External services** such as email APIs may be integrated for appointment notifications, billing alerts, password recovery, and communication support.
- The system supports **secure communication** between frontend and backend components using JSON-based REST APIs, ensuring efficient data exchange and system interoperability.
- The architecture is designed with **modular components** so that future integrations, upgrades, and additional AI functionalities can be added with minimal impact on existing system operations.

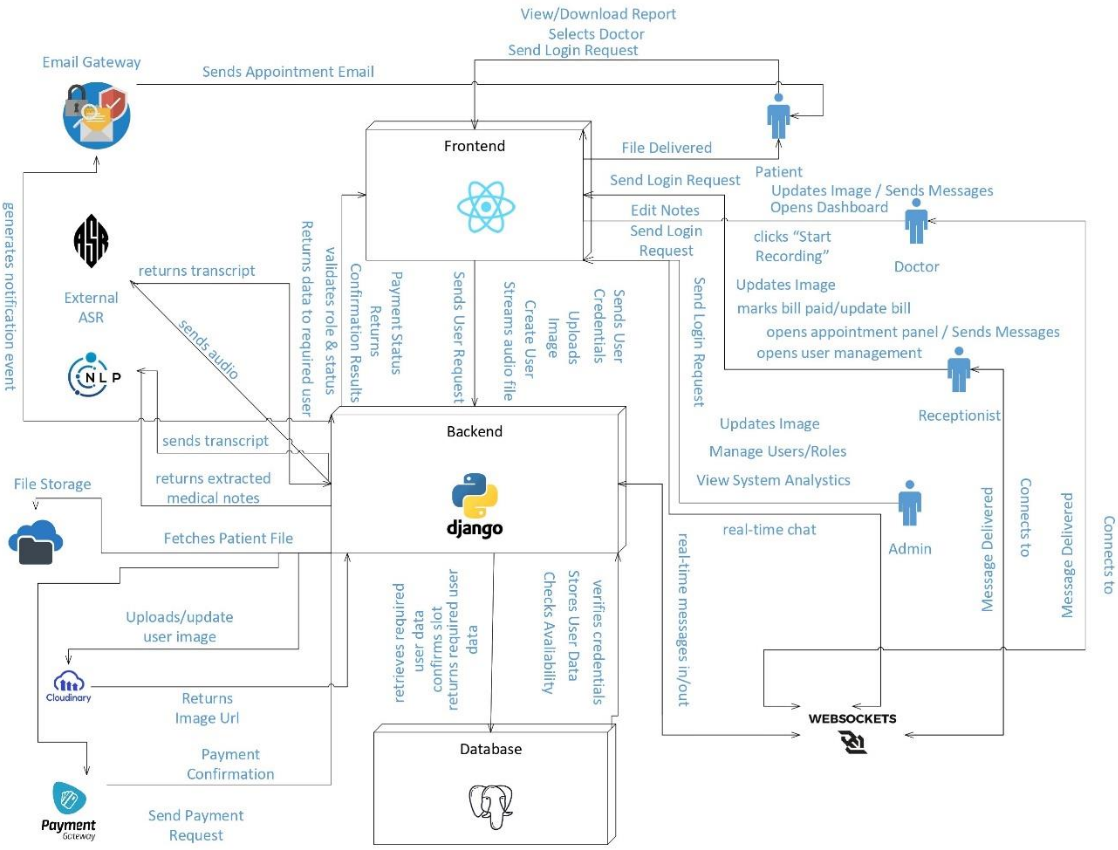


Figure 7.1 System Architecture Diagram

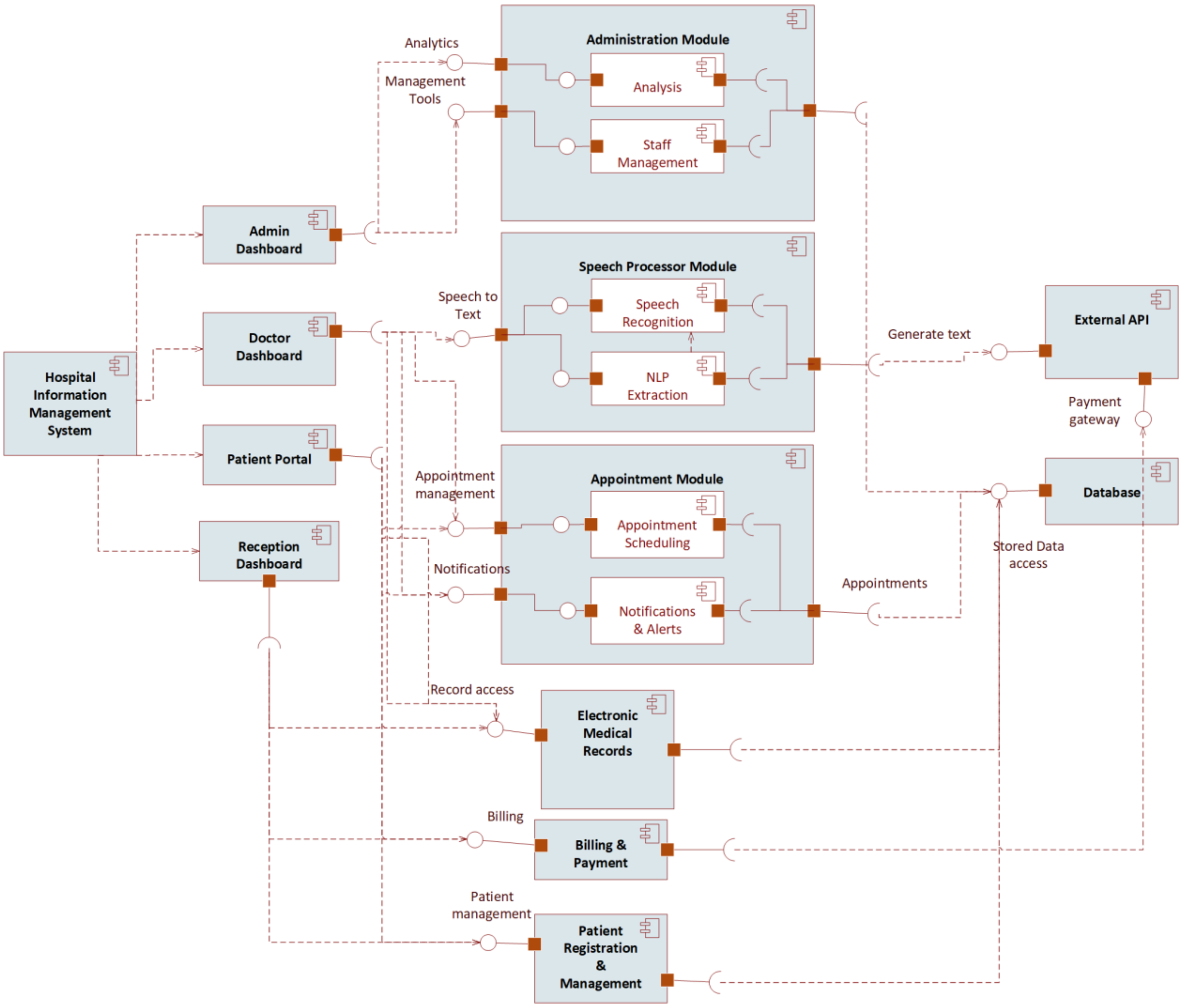


Figure 7.2 Component Diagram

7.3 Module Implementations

7.3.1 AI Consultation Recording & Documentation Module

Flow:

• Doctor-patient conversation is captured through microphone input →
• Speech-to-text processing converts conversation into text →
• NLP-based processing extracts important medical information such as symptoms, diagnoses, medications, and recommendations →
• Structured medical reports and SOAP notes are generated →
• Doctor reviews, edits if necessary, and confirms the generated notes →
• Verified records are stored in the PostgreSQL database.

Features:

• Voice-based consultation recording
• Automatic SOAP note & medical report generation
• Doctor validation before saving

• Secure storage of consultation history
• Support for English, Urdu, and Punjabi languages

UI Elements:

• Voice recording controls
• Consultation history panel
• AI-generated summary preview
• SOAP note editor and confirmation interface

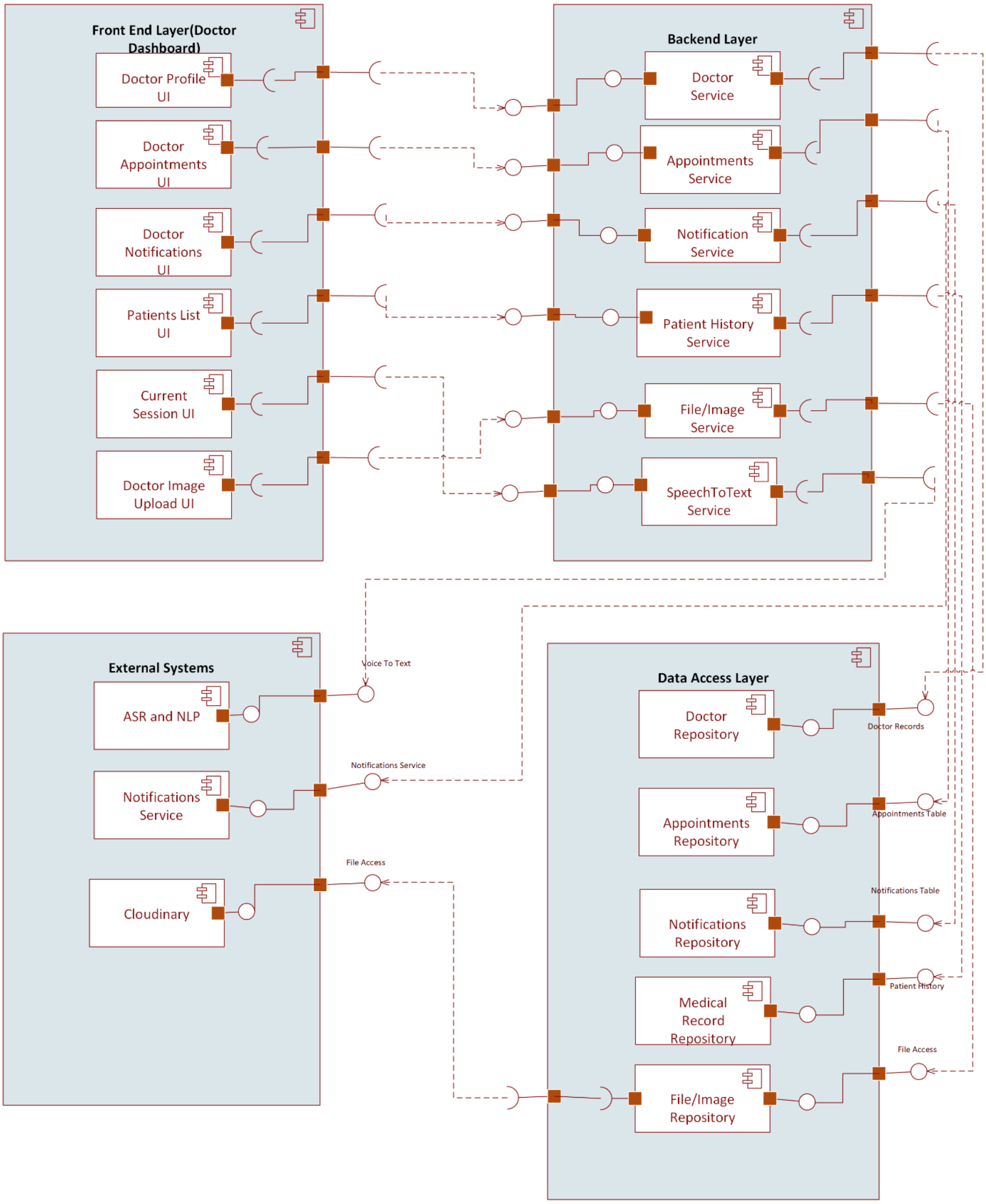


Figure 7.3.1 Doctor Component Diagram

7.3.2 Appointment & Queue Management Module

Flow:

• Patient requests appointment →
• Receptionist or patient schedules appointment →
• System assigns queue position on a first-come-first-serve basis →
• Real-time queue status is updated dynamically →
• Patient receives appointment and queue information through dashboard or notifications.

Features:

• Online appointment booking
• Real-time queue tracking
• Doctor-specific appointment management
• Queue status updates
• Appointment history management

UI Elements:

• Appointment booking form
• Queue status display
• Doctor schedule panel
• Receptionist management dashboard

7.3.3 Patient Record & Prescription Management Module

Flow:

• Doctor verifies consultation notes →
• Medical reports and prescriptions are stored in the database →
• Authorized users retrieve patient history when required →
• Patients can access reports and prescriptions through the patient portal.

Features:

• Centralized patient medical records
• Prescription history management
• Downloadable medical reports
• Doctor-specific treatment history
• Secure record access

UI Elements:

• Patient profile dashboard
• Prescription viewer
• Medical history tables
• Report download interface

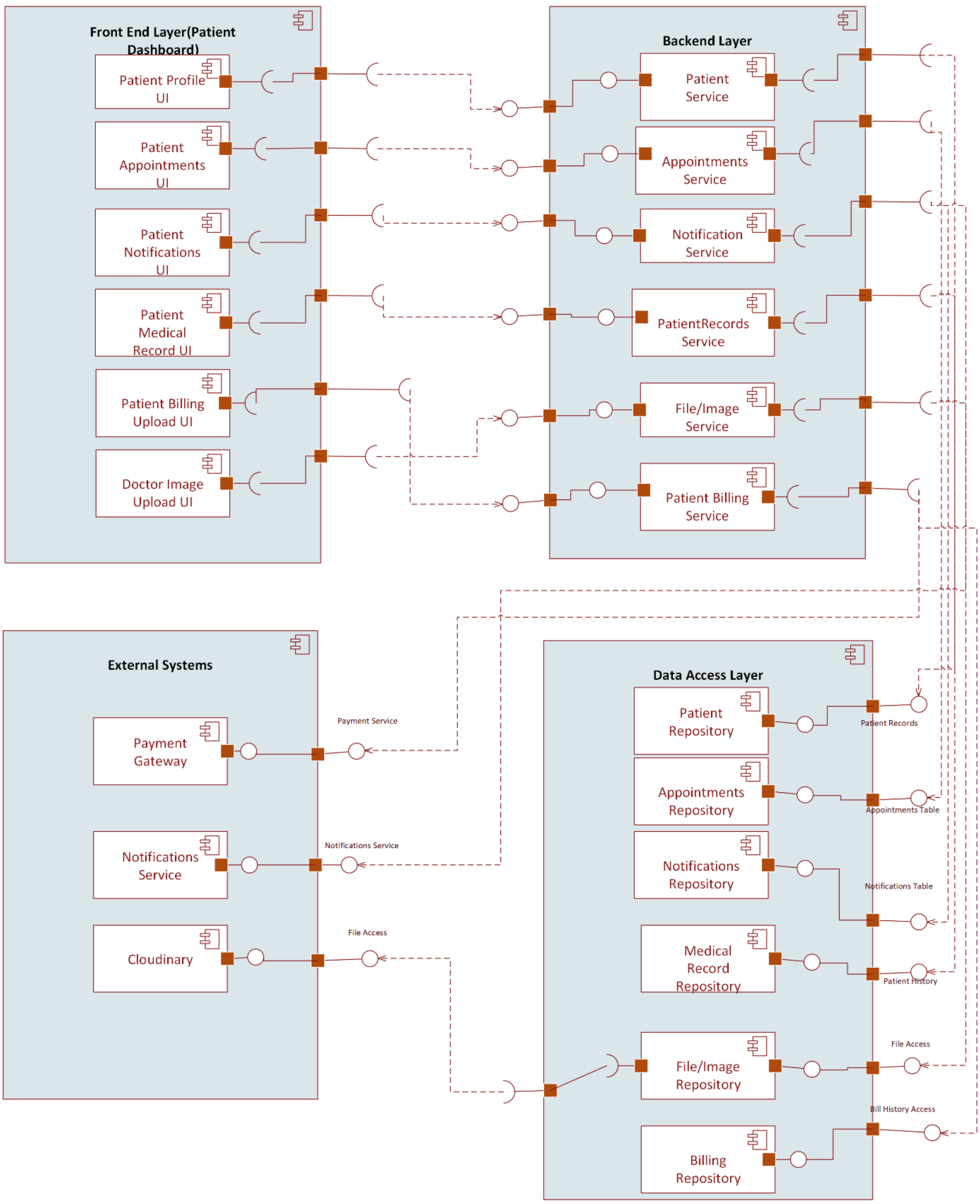


Figure 7.3.2 Patient Component Diagram

7.3.4 Communication & Notification Module

Flow:

• Messages or notifications are generated by the system or healthcare staff →
• Notifications are sent through dashboard alerts or email services →
• Users can view communication history and updates within the system.

Features:

• Secure healthcare-related messaging
• Appointment notifications and reminders
• Billing and report notifications
• Follow-up communication support

UI Elements:

• Messaging interface
• Notification panel
• Conversation history section

7.3.5 Administrative Dashboard Module

Features:

• User management and account approval
• Doctor and receptionist assignment management
• Hospital activity monitoring
• Patient and appointment analytics
• Billing and operational management

UI Elements:

• Analytics dashboard
• User management tables
• Activity monitoring panels
• Billing and reporting sections

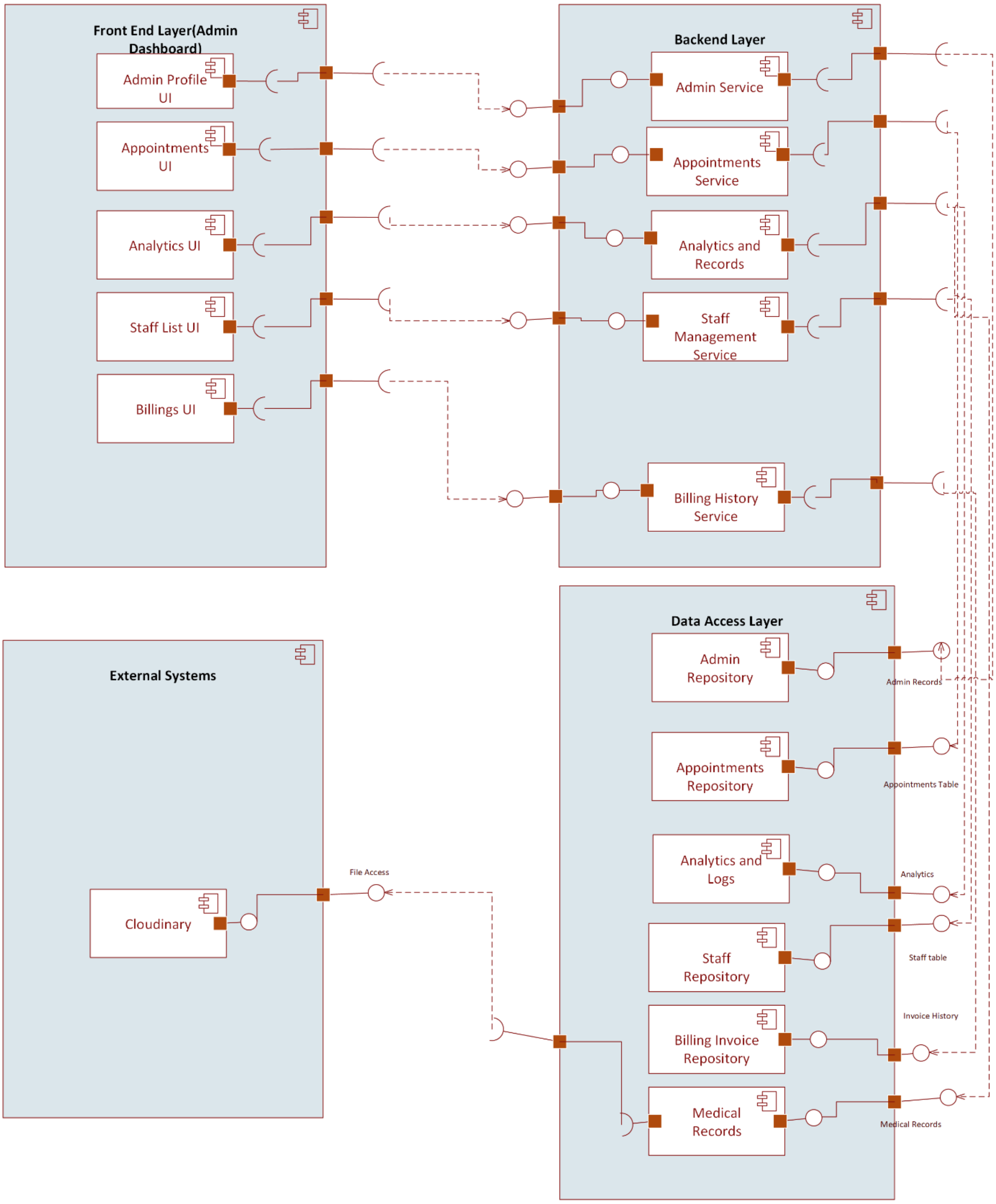


Figure 7.3.3 Admin Component Diagram

7.3.6 Authentication & Access Control Module

Features:

• Secure login and authentication
• Role-Based Access Control (RBAC)
• Controlled patient account approval
• Session management and access restrictions

Supported Roles:

• Administrator
• Doctor
• Receptionist
• Patient

UI Elements:

• Login and registration forms
• Account approval dashboard
• Password recovery interface
• Role management controls

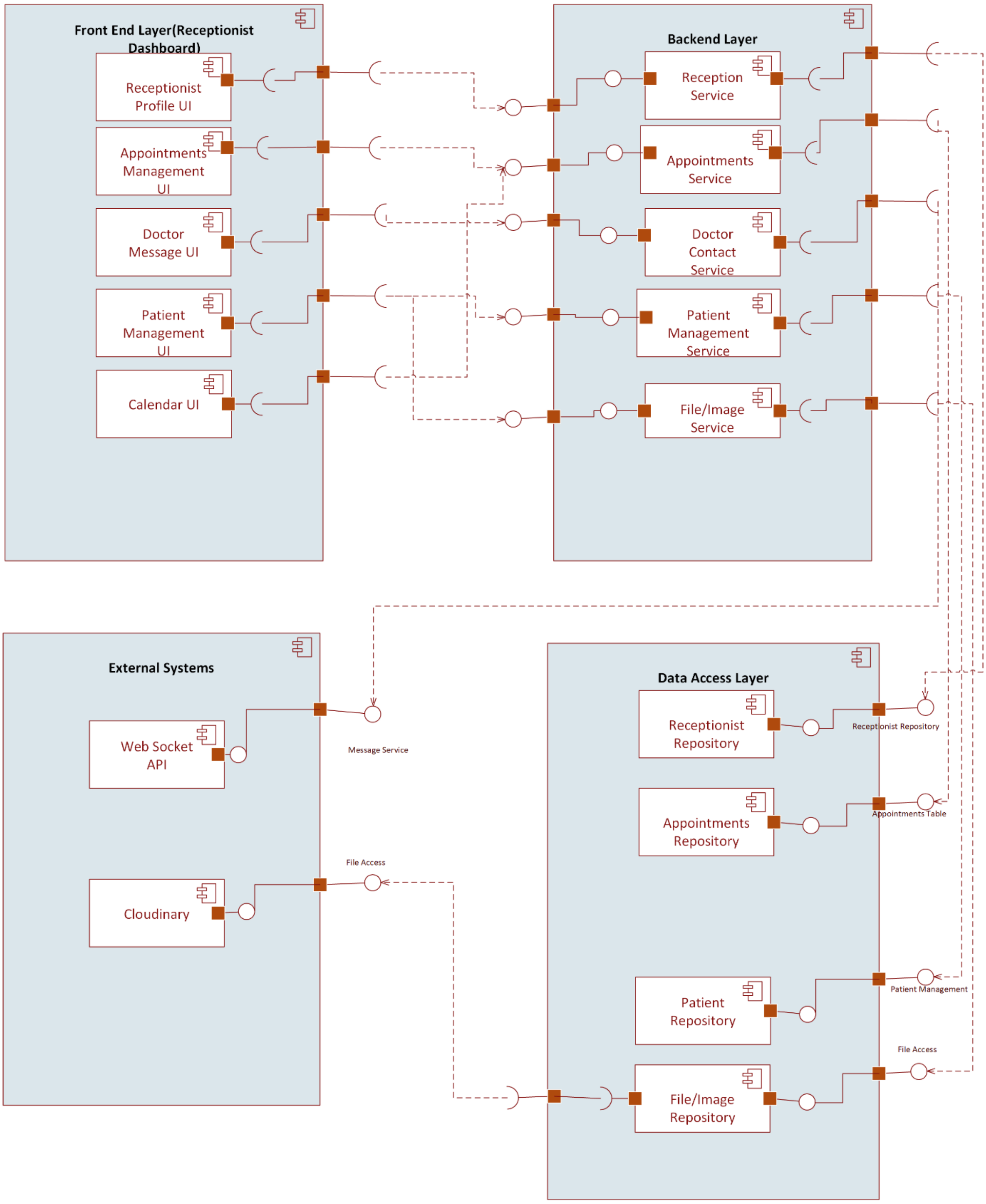


Figure 7.3.4 Receptionist Component Diagram

7.4 Deployment Strategy

7.4.1 Frontend Application

- Doctors, administrators, receptionists, and patients deploy the ReactJS frontend dashboard on a web hosting platform (Netlify).

- The frontend communicates securely with backend APIs for real-time hospital operations and data management.

7.4.2 Backend Service

- Backend services manage authentication, hospital workflows, database communication, AI-assisted documentation, and REST API processing.
- Environment variables are used for secure configuration of database credentials, API keys, and system settings.

7.4.3 Database Deployment

- PostgreSQL is used as the central database system for storing patient records, appointments, prescriptions, billing information, and system logs.
- Regular backup mechanisms are maintained to ensure data recovery and reliability.

7.4.4 AI & Voice Processing Services

- Speech recognition and NLP processing modules are integrated with the backend system for voice-to-text conversion and structured medical report generation.
- AI processing components communicate securely with the backend through API-based integration.

7.4.5 File & Report Management

- Medical reports, prescriptions, and uploaded documents are stored using secure local or cloud-based storage systems.
- PDF generation services are integrated for downloadable reports and prescriptions.

7.5 Code/API Documentation

- Django REST Framework APIs are documented for frontend-backend communication and system integration.
- Internal project documentation and module descriptions are maintained through GitHub repositories and README files.
- API endpoints include proper request/response structures and authentication guidelines for developers and future maintenance.
- System architecture, database schema, and workflow documentation are maintained to support future upgrades and scalability.

Chapter 08: Design Document

8.1 State Transition Diagram

Figure 8.1 State Transition Diagram

8.2.1 DFD Level 0

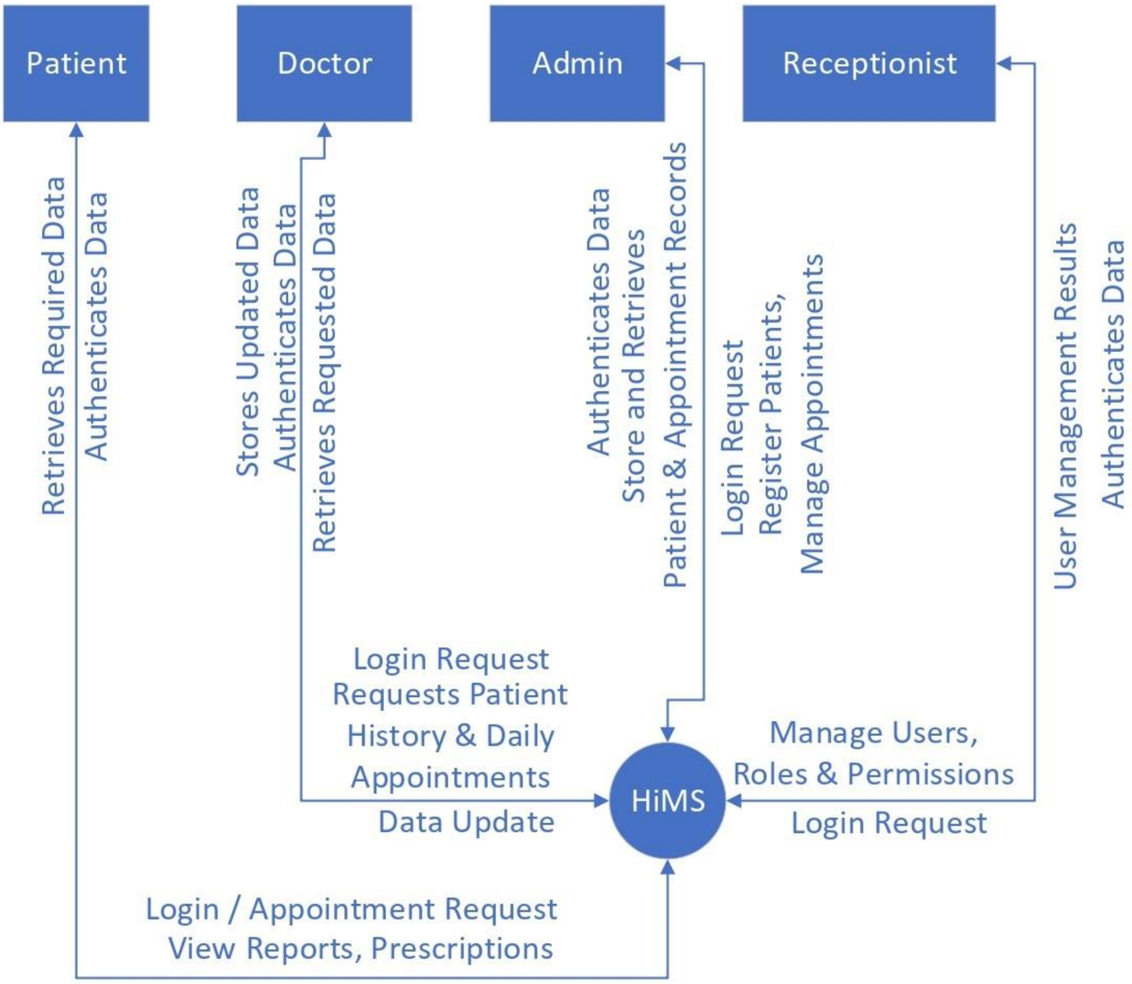


Figure 8.2.1 DFD Diagram (Level 0)

8.2.2 DFD Level 1

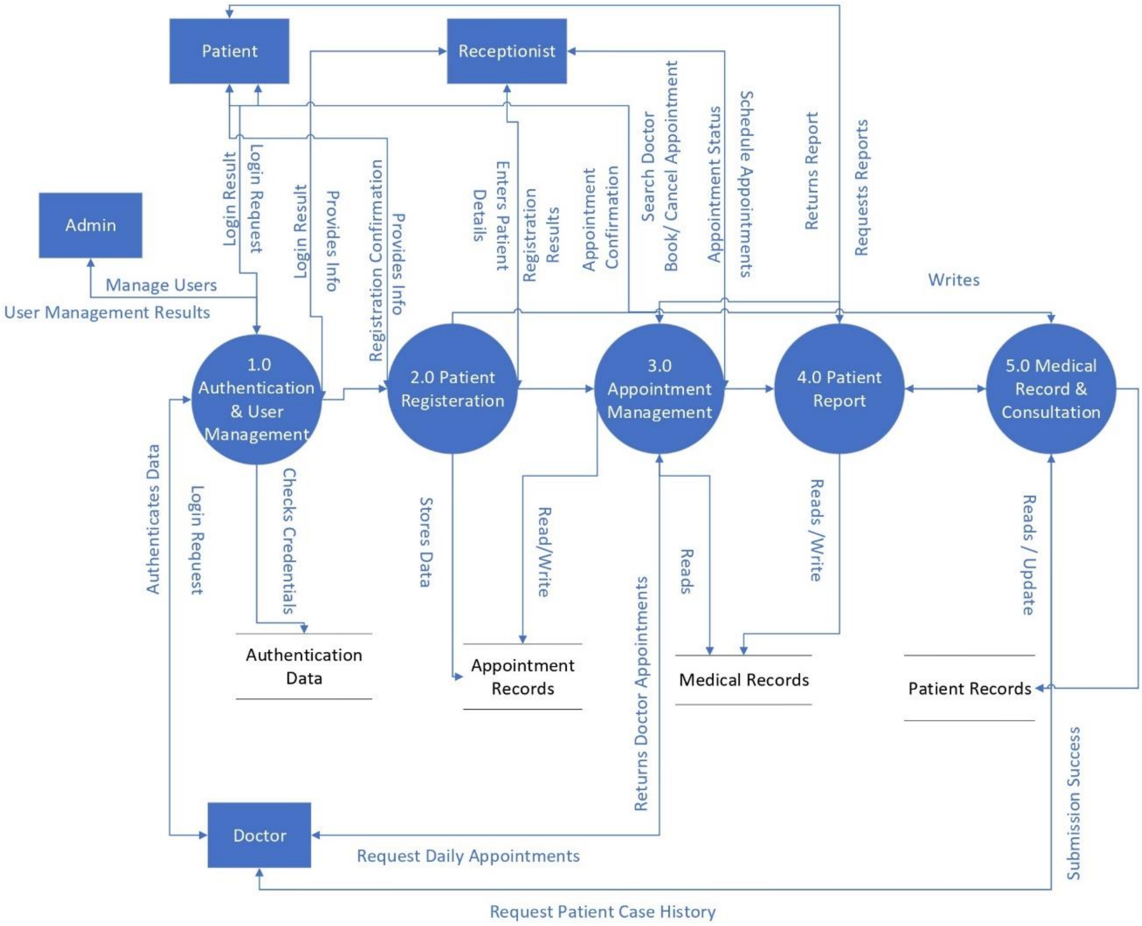


Figure 8.2.2 DFD Diagram (Level 1)

8.2.3 DFD Level 2

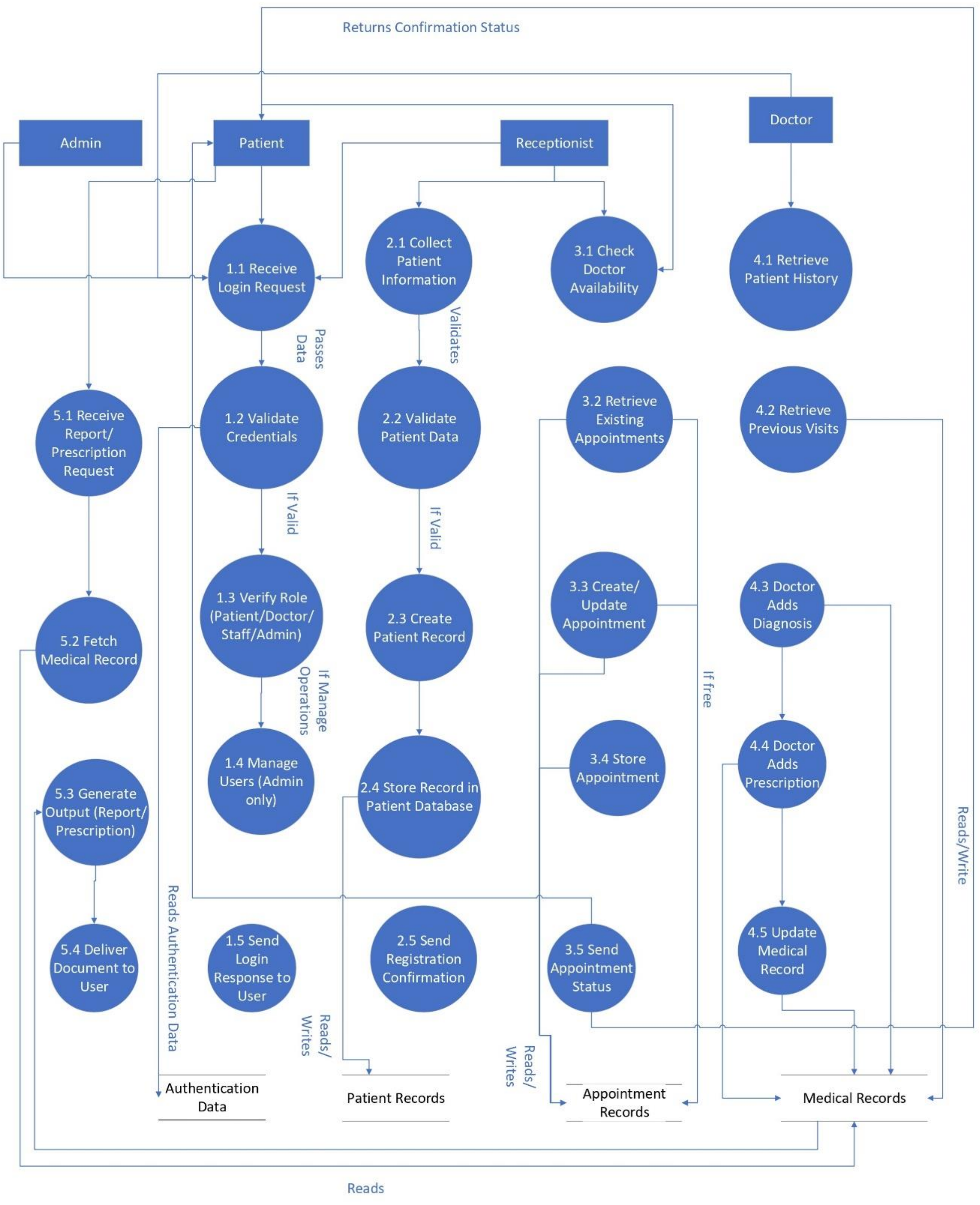


Figure 8.2.3 DFD Diagram (Level 2)

8.3 Sequence Diagram

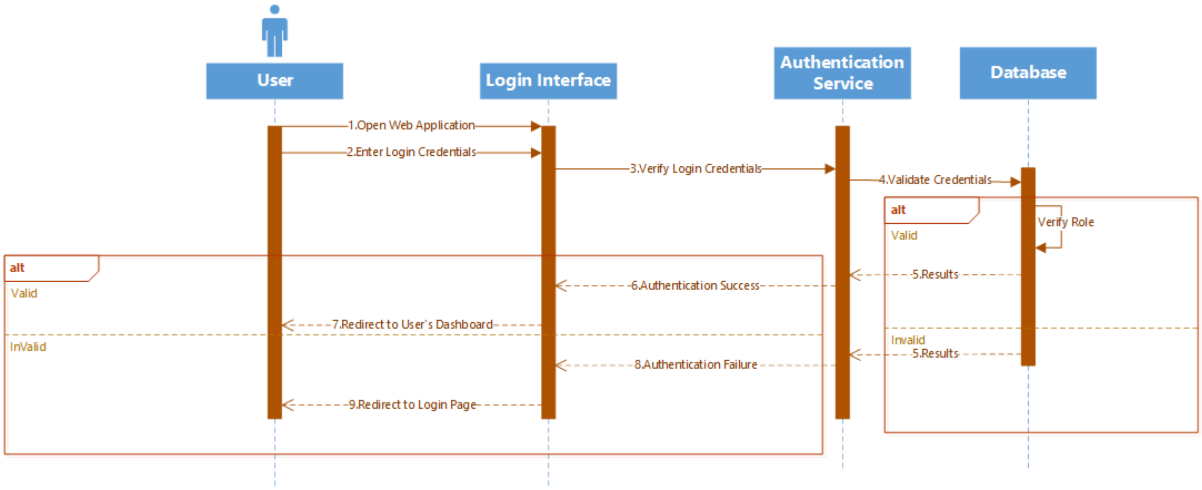


Figure 8.3.1 Sequence Diagram (Login)

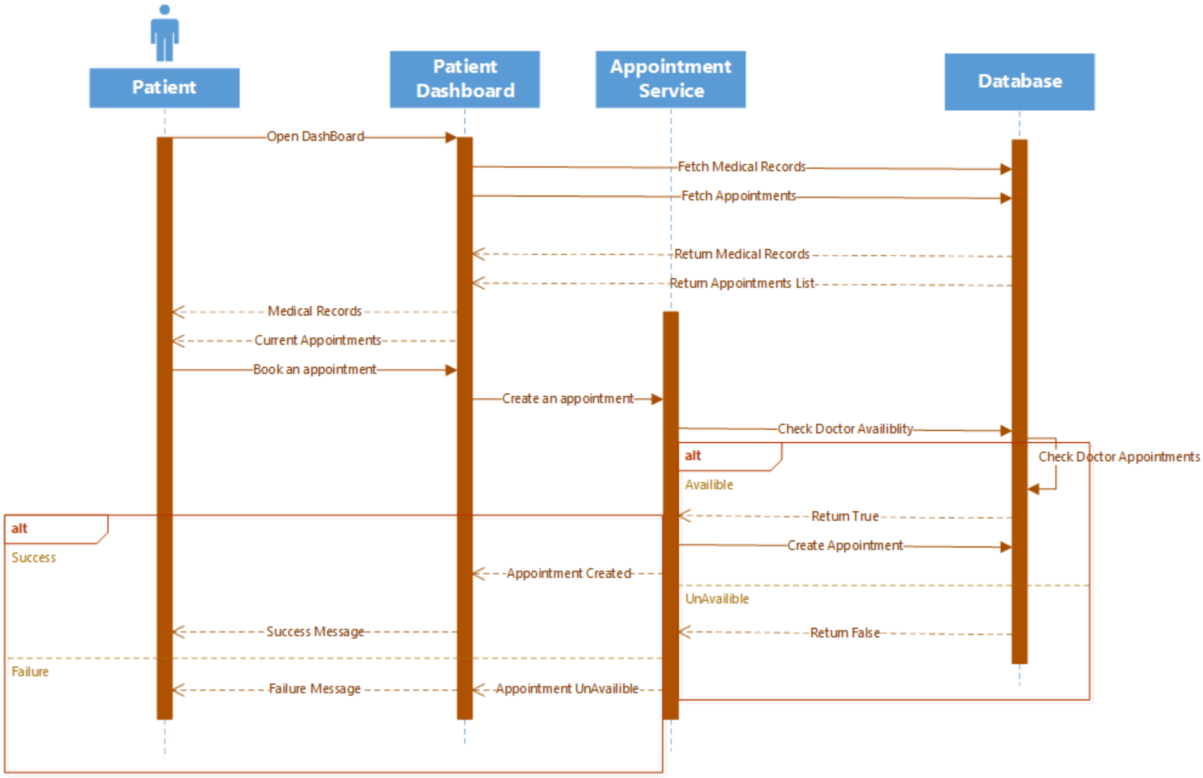


Figure 8.3.2 Sequence Diagram (Patient)

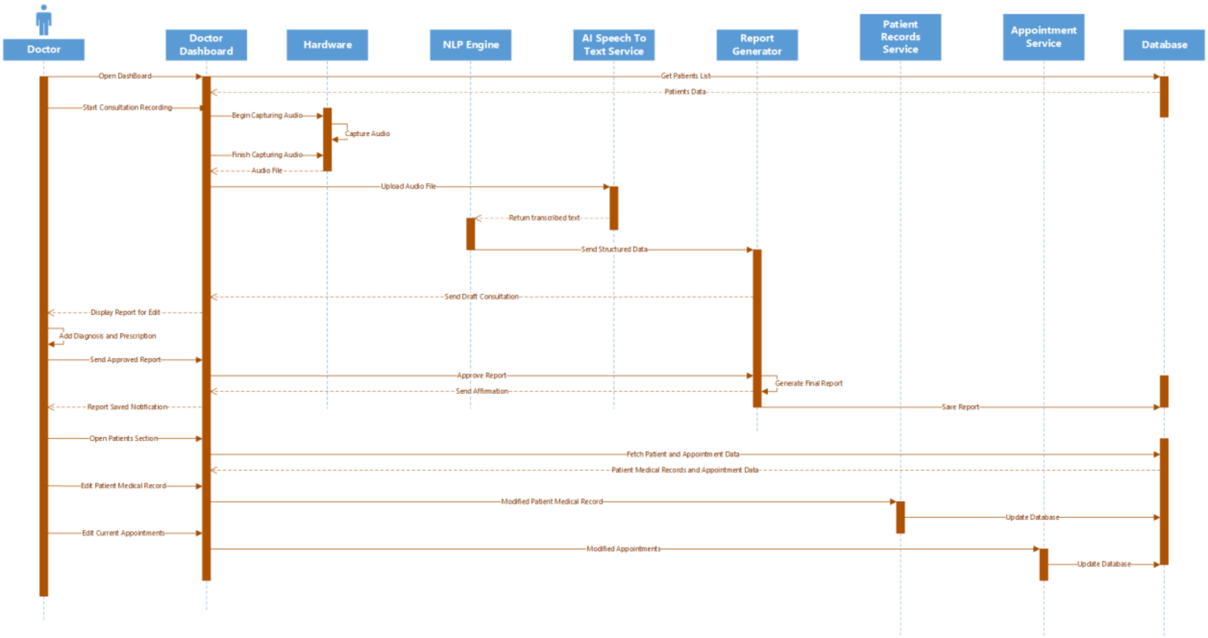


Figure 8.3.3 Sequence Diagram (Doctor)

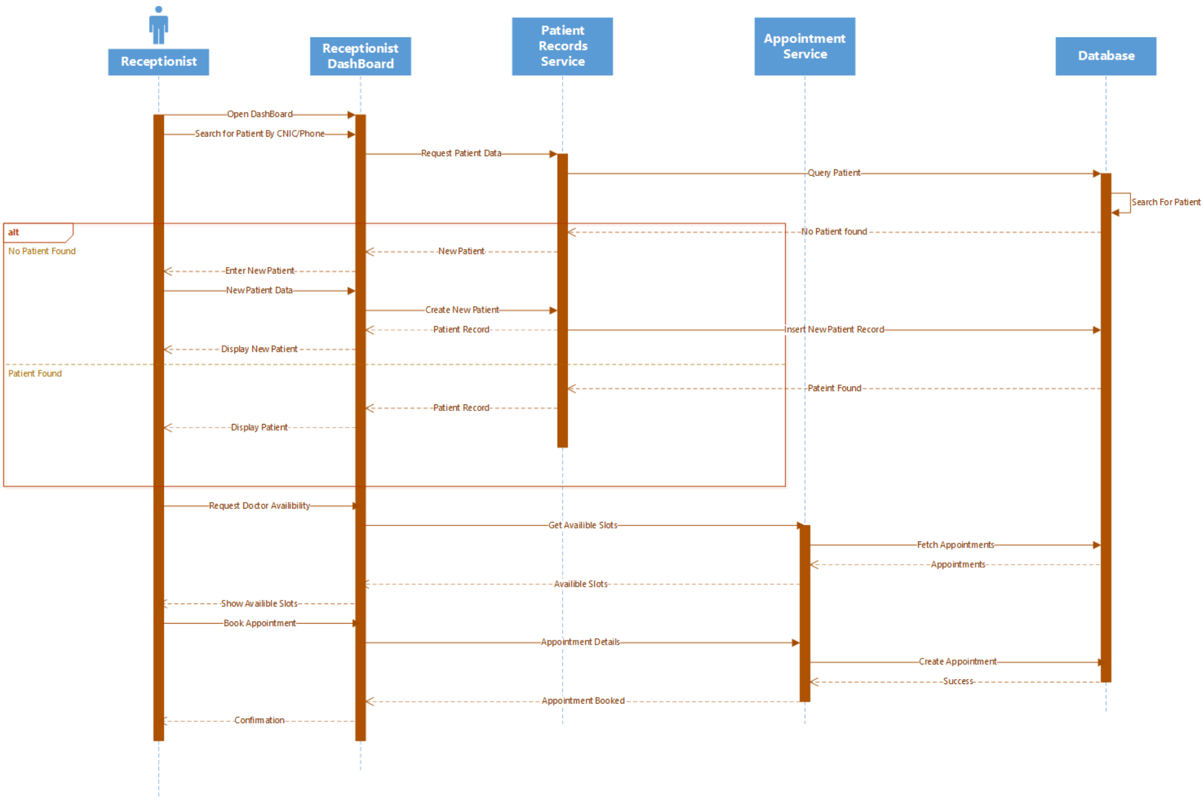


Figure 8.3.4 Sequence Diagram (Receptionist)

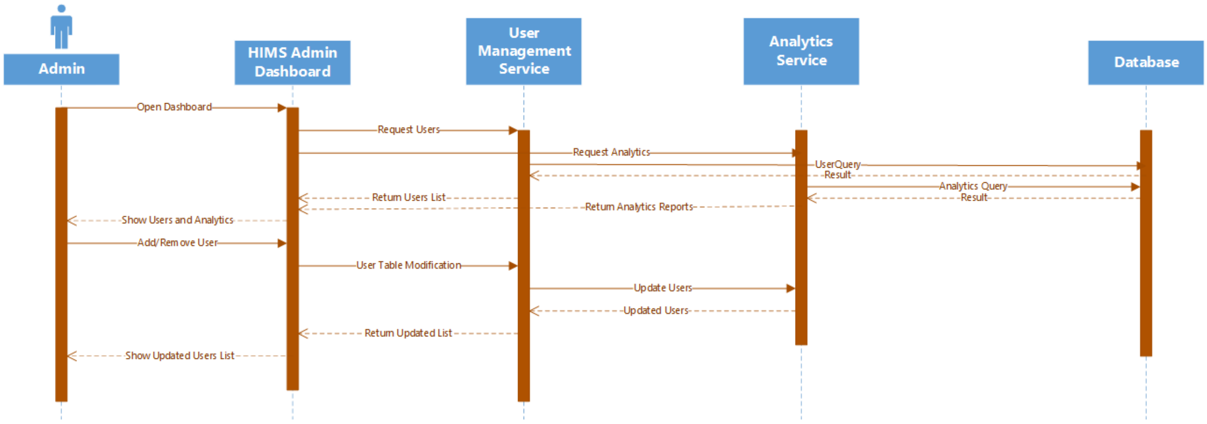


Figure 8.3.5 Sequence Diagram (Admin)

8.4 ER Diagram

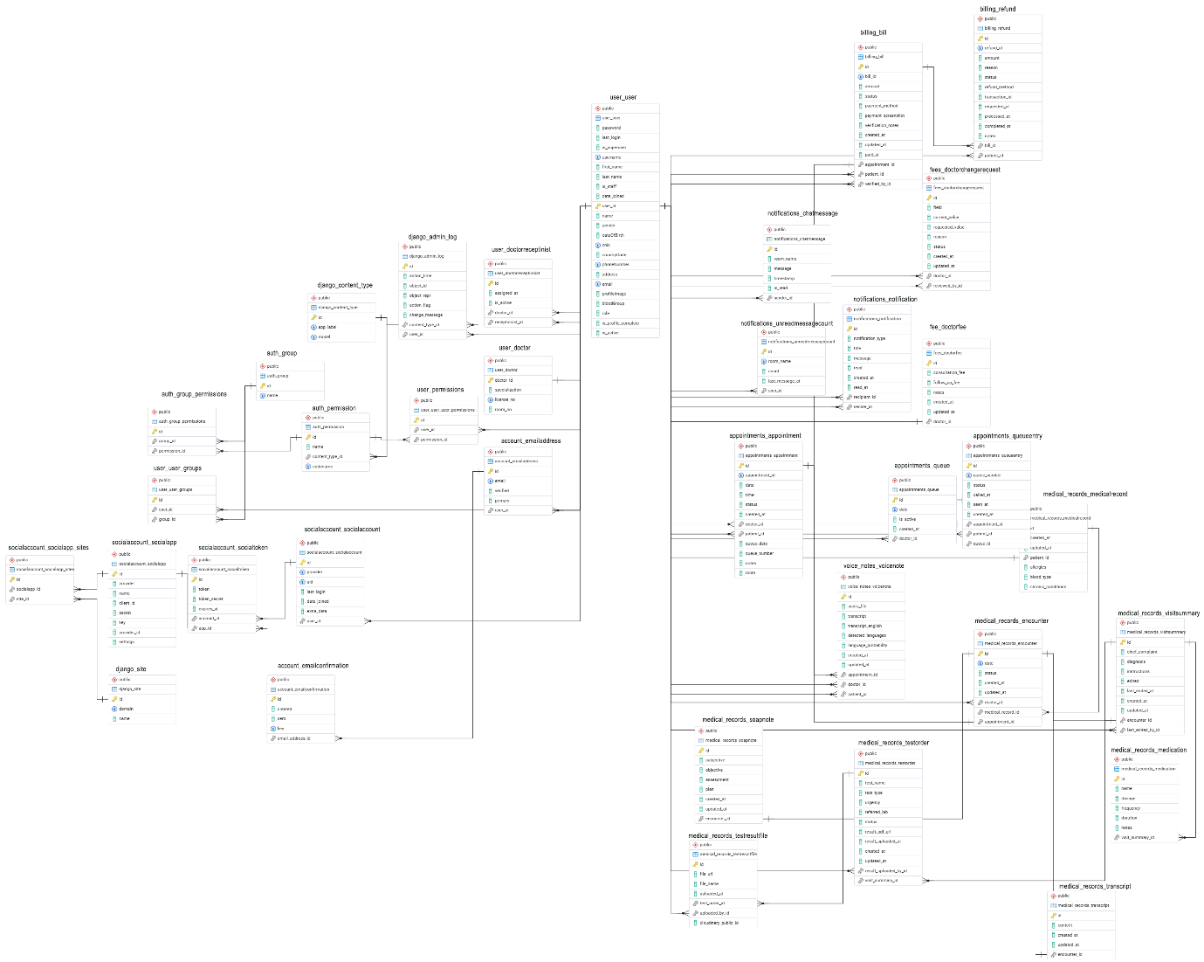


Figure 8.4 ER Diagram

8.5 Class Diagram

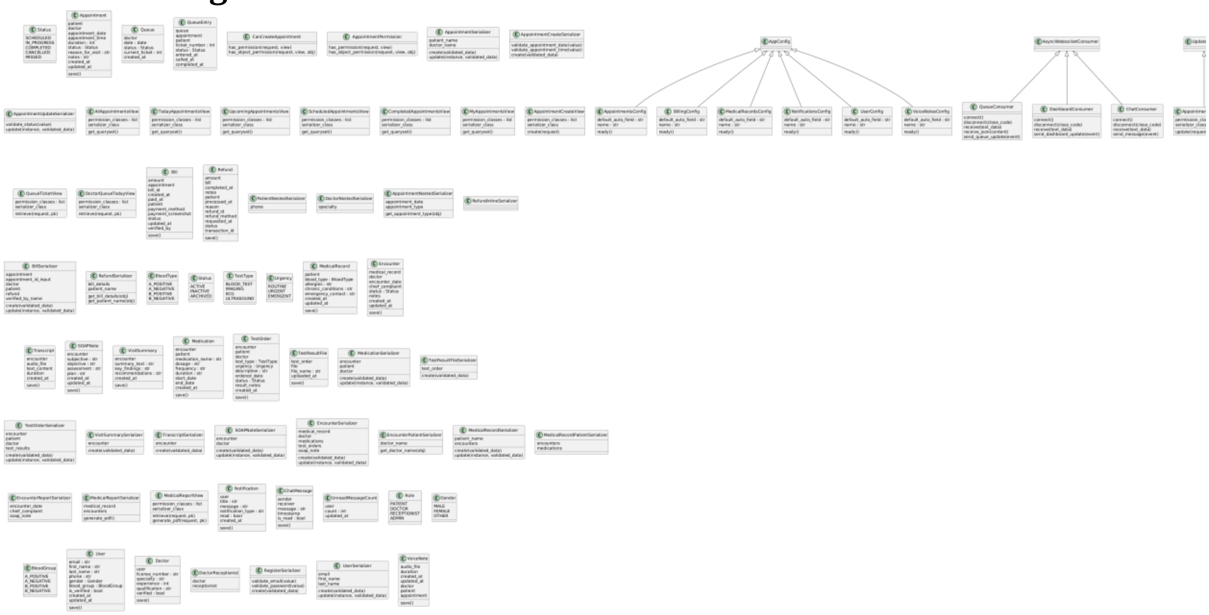


Figure 8.5.1 HIMS Class Diagram

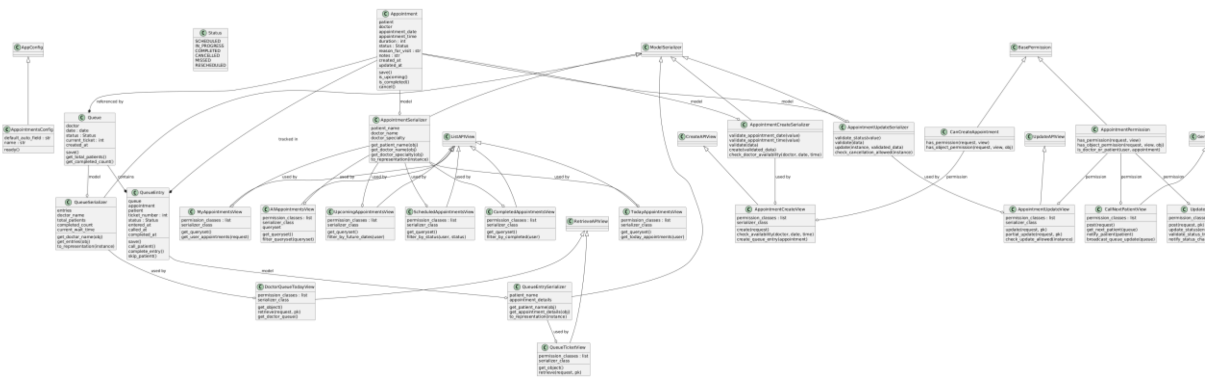


Figure 8.5.2 Appointment Class Diagram

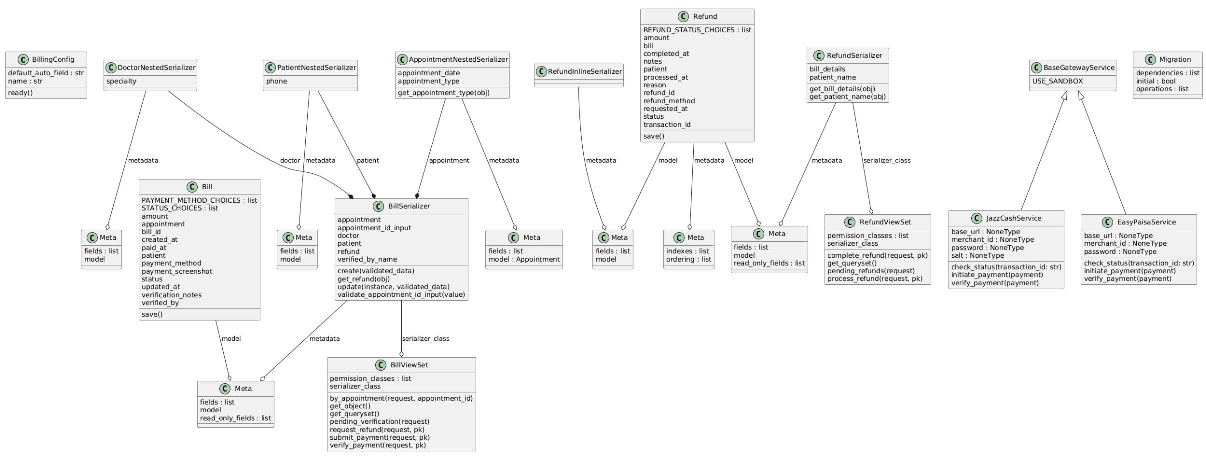


Figure 8.5.3 Billing Class Diagram

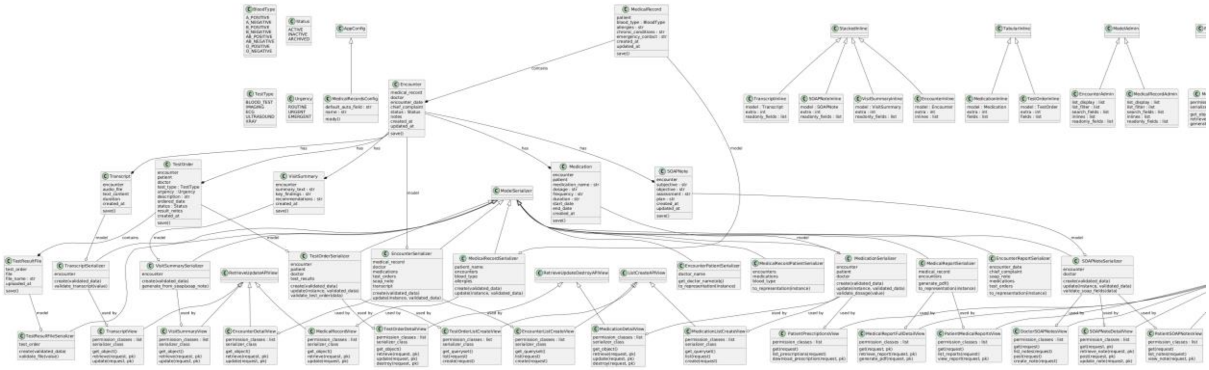


Figure 8.5.4 Medical Record Class Diagram

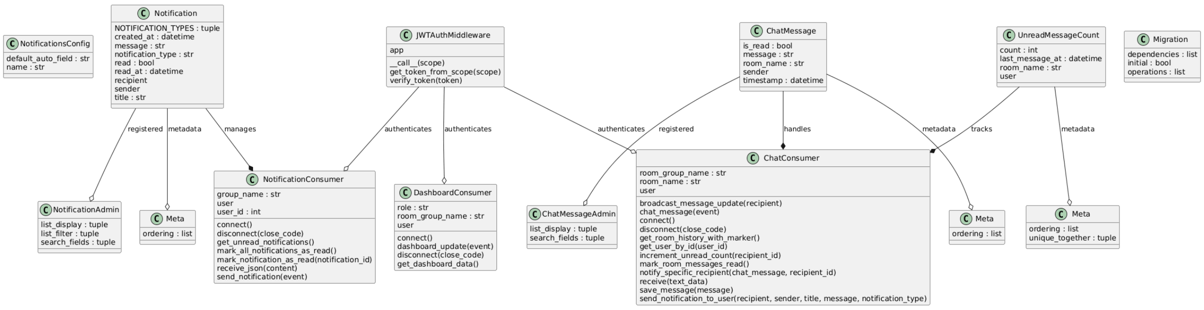


Figure 8.5.5 Notification Class Diagram

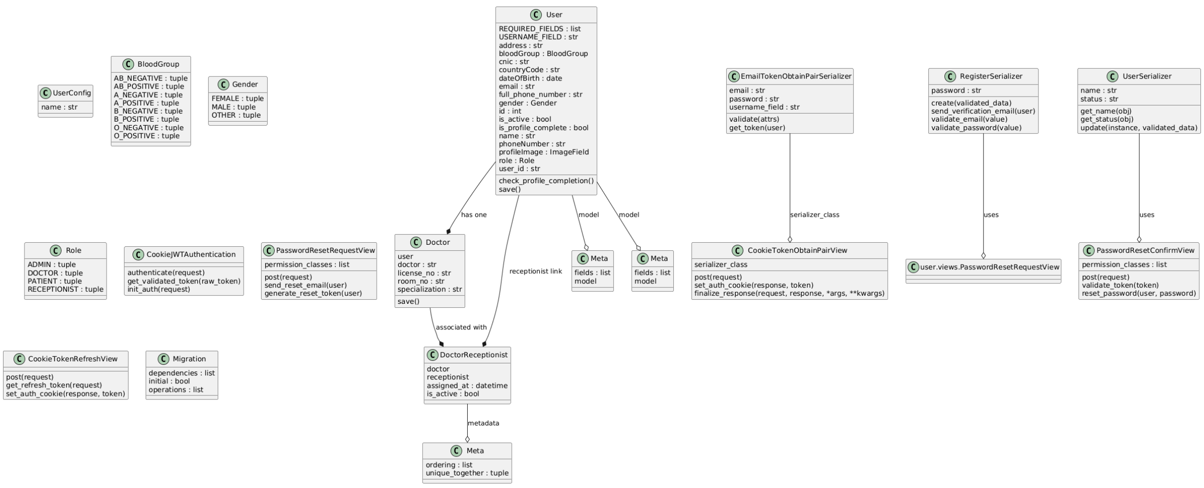


Figure 8.5.6 User Class Diagram

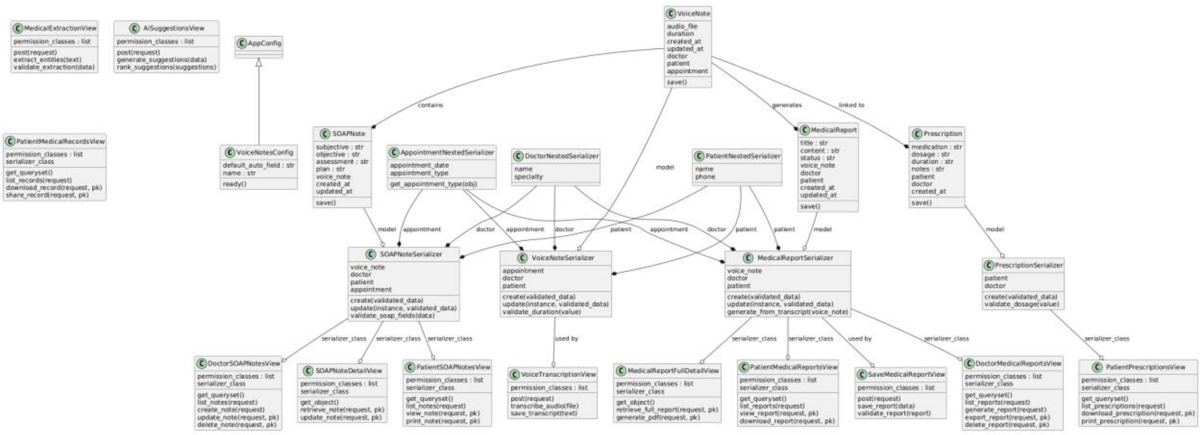


Figure 8.5.7 Voice Notes Class Diagram

Chapter 09: UI Interfaces

9.1 Landing Page

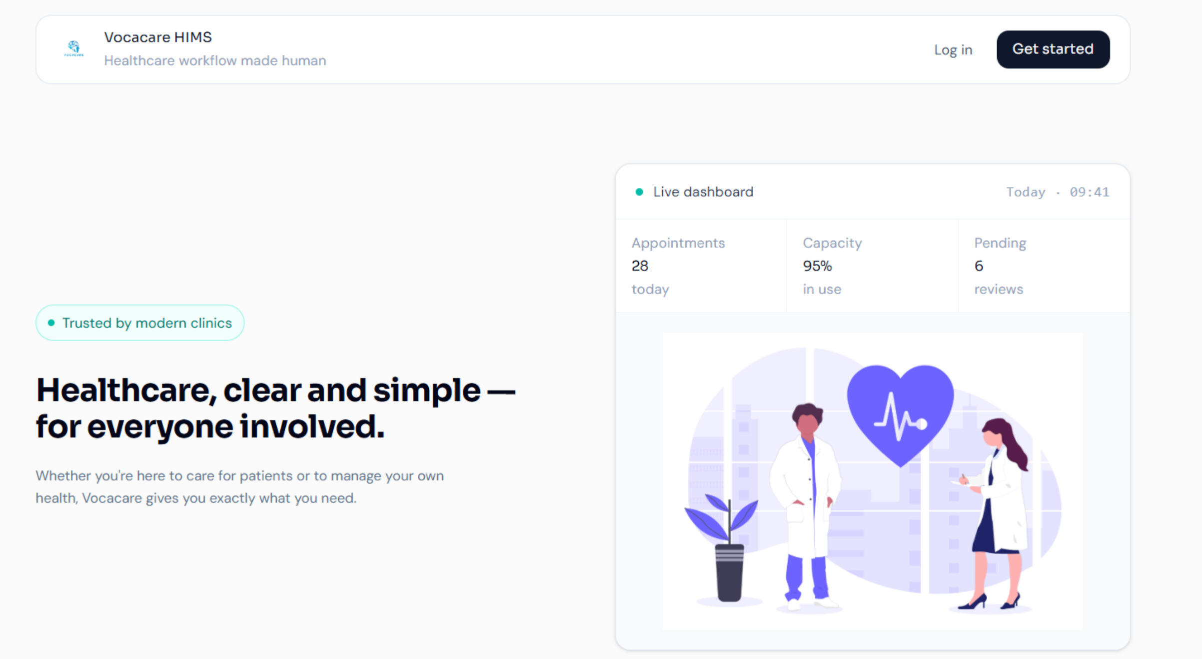


Figure 9.1 Landing Page

Purpose & Context:

The Landing Page serves as the primary entry point and first visual impression of the Vocacare system for all user types — doctors, patients, receptionists, and administrators. The page directs users to either register a new account or log into an existing one. No authentication is required to view the landing page, making it fully public facing.

Element Details:

1. Branding Area

- Vocacare logo displayed prominently in the header
- Tagline: "Healthcare, clear and simple – for everyone involved."

2. Primary Actions

- **Sign In Button** → Navigates to the Registration Page where new users can create a Vocacare account by providing their name, email, and password. Patient accounts require admin approval before activation.
- **Log In Button** → Navigates to the Login Page where existing users authenticate using their registered email and password. Upon successful authentication, users are redirected to their role-specific dashboard.

User Flow (Landing Page)

1. **User clicks "Sign In"**

   System Action: Performs a smooth client-side navigation to the Registration Page. The registration form renders with fields for full name, email address, and password. A notice informs the user that patient accounts require administrative approval before activation.

2. **User clicks "Log In"**

   System Action: Performs a smooth client-side navigation to the Login Page. The login form renders with email and password fields, along with a "Log in with Google" option. Upon successful authentication, the backend validates the user's role and redirects to the corresponding dashboard.

3. **User returns to landing**

   System Action: No session or authentication state is retained on the landing page. Navigating back to the root URL results in a fresh render of the landing page with default state. If the user is already authenticated, the system automatically redirects them to their role-specific dashboard instead of showing the landing page again.

9.2 Register Page

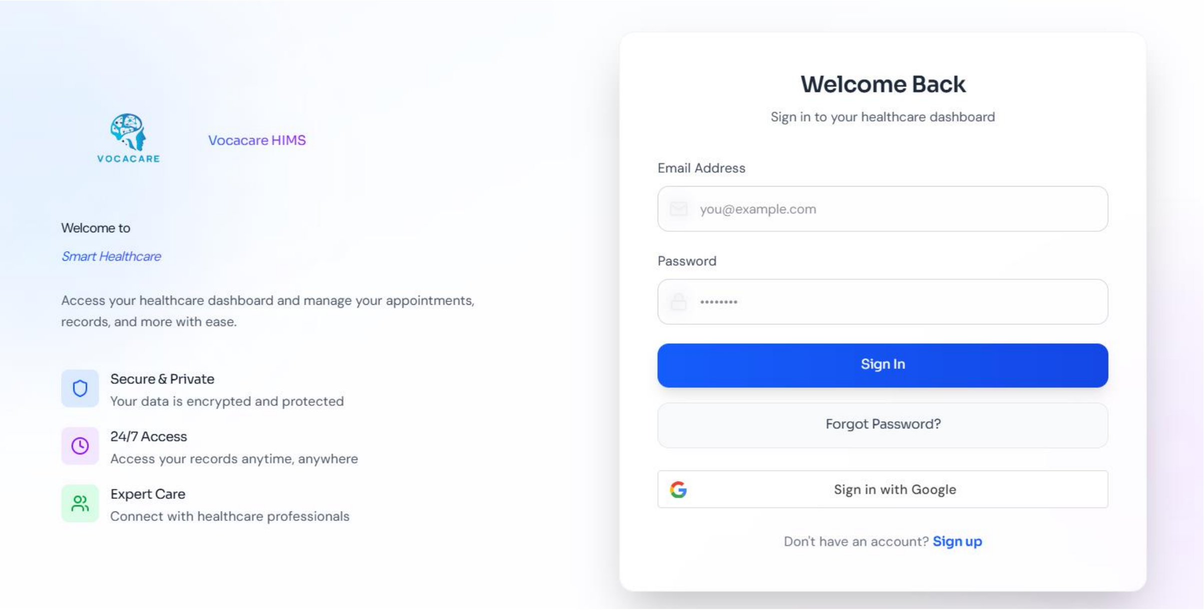


Figure 9.2 Register Page

Purpose & Context:

The Registration Page allows new users to create an account. It is accessible to patients who self-register, as well as to staff accounts created through the admin panel. The form collects essential identity and credential information. Patient accounts submitted through this page remain inactive until reviewed and approved by an authorized administrator, ensuring only verified individuals gain system access.

Element Details:

Register Form

- Full Name input field (required)
- Email Address input field with format validation (required)
- Password input field with visibility toggle (minimum 8 characters)
- Confirm Password field to prevent entry errors
- Register Button — submits the form and triggers the account creation API
- Link to Login Page for users who already have an account

User Flow

1. User fills in name, email, and password → Clicks Register System Actions:

- Validates that all required fields are filled
- Validates email format using regex
- Checks that password meets minimum length requirements and matches the confirm password field
- Sends credentials to the registration API endpoint
- If successful → displays a confirmation message: "Account created. Awaiting admin approval." (for patient role)
- If email already exists → shows inline error: "An account with this email already exists."
- If passwords do not match → shows inline error: "Passwords do not match."

9.3 Login Page

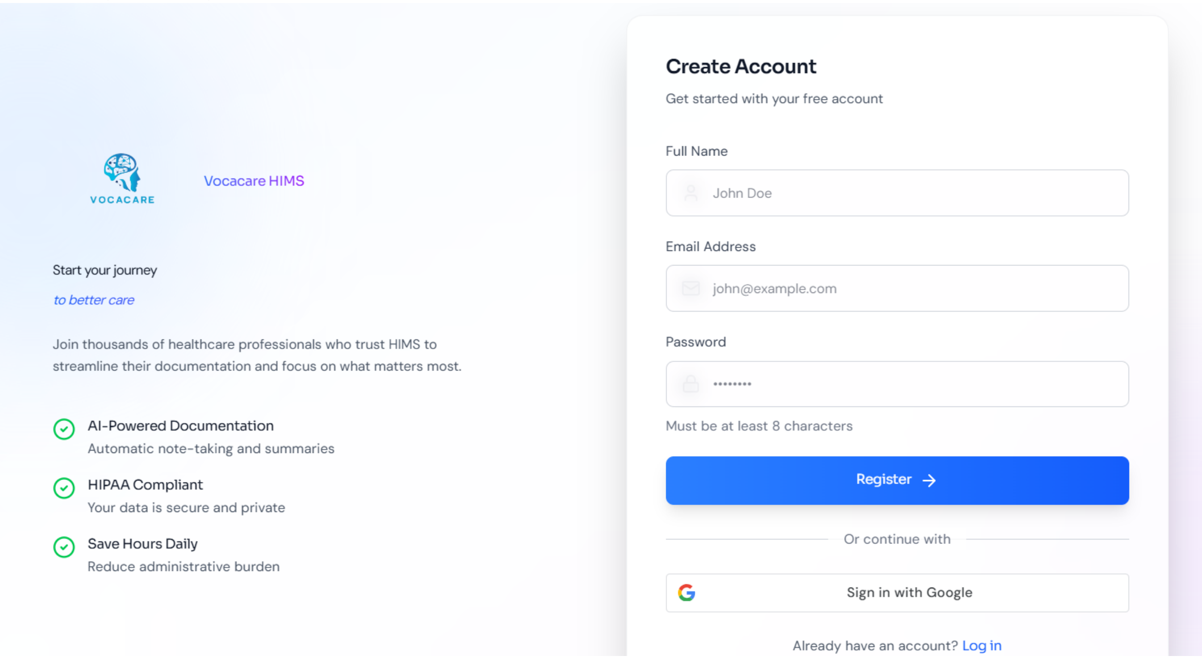


Figure 9.3 Login Page

Purpose & Context:

The Login Page provides secure, role-aware authentication for all system users. It supports both standard email/password login and Google OAuth sign-in. Upon successful authentication, the backend identifies the user's assigned role and redirects them directly to their corresponding dashboard — eliminating manual navigation. Failed login attempts display clear, inline error messages to guide the user.

Element Details:

Login Form

- Email input field with format validation
- Password input field with show/hide toggle
- Sign In button — submits credentials to the authentication API
- "Log in with Google" button — initiates OAuth 2.0 Google sign-in flow
- Inline error message area for invalid credentials
- "Forgot Password?" link for password recovery flow
- Link to Register Page for new users

User Flow

1. Enter Email + Password → Click Sign In System Actions:

- Validates email format on the frontend before submission
- Validates that password field is not empty
- Sends credentials to the authentication API
- If valid → backend returns a session token; frontend stores it in an HTTP-only cookie and redirects the user to their role-specific dashboard (Doctor, Patient, Receptionist, or Admin)
- If invalid → displays inline error: "Incorrect email or password."
- If account is pending approval → displays: "Your account is awaiting admin approval."

2. Enter Name + Email + Password → Click Register System Actions:

- Validates all fields client-side before submission
- Sends registration data to the API
- If valid → creates account and shows confirmation message
- If invalid → shows inline error: "Invalid email or password format."

9.3 Dashboard

9.3.1 Patient Dashboard

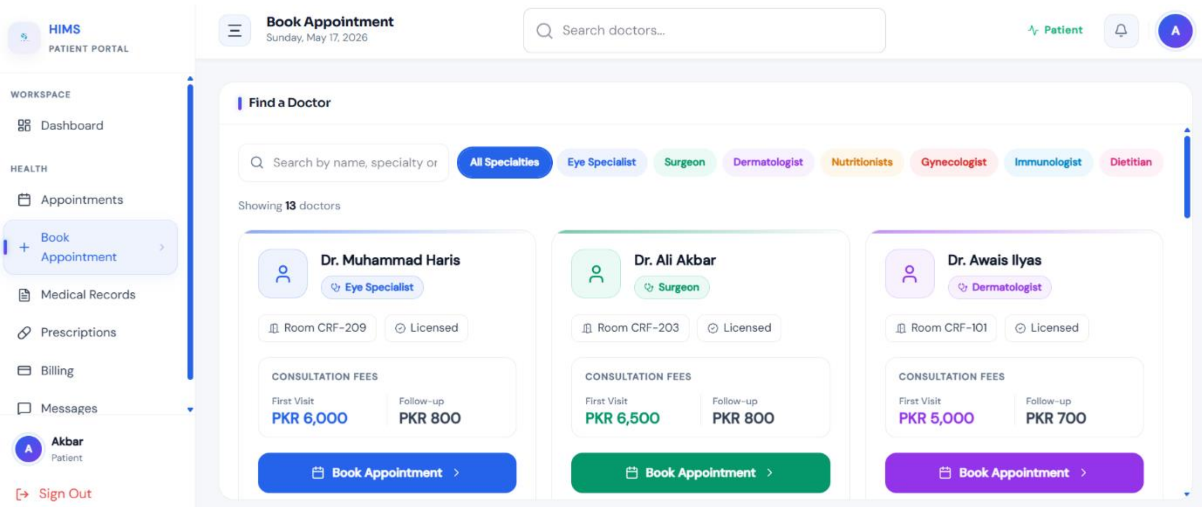


Figure 9.4 Patient Dashboard

Purpose & Context:

The Patient Dashboard is the central interface for patients to manage their healthcare interactions with the hospital. It provides a unified, easy-to-navigate view of appointments, medical history, prescriptions, and billing information. The dashboard is designed to be accessible to users with varying levels of technical familiarity, offering clear action buttons and organized module cards. All data displayed is personal to the authenticated patient and retrieved securely from the backend.

Element Details:

Header

- Vocacare logo and platform name
- Personalized welcome message (e.g., "Welcome, [Patient Name]")
- Global search bar for finding doctors or appointment records
- Notification bell with dropdown for unread alerts
- User profile avatar with dropdown menu (Profile, Settings, Logout)

Modules

- **Upcoming Appointments** — displays scheduled appointments with doctor name, date, time, and queue position; allows cancellation
- **Medical History** — chronological timeline of past consultations with expandable detail views for diagnoses
- **Prescriptions** — list of active and past prescriptions with download option as PDF
- **Billing Summary** — overview of pending and paid invoices with payment action buttons

- **Quick Actions** — shortcut buttons for Book Appointment, View Reports, Contact Reception

User Flow

1. **Patient books an appointment**:

   System Action: Opens the appointment booking panel with a calendar view. Patient selects a doctor and available date slot. System checks for conflicts and confirms the slot. Appointment appears in the Upcoming Appointments module with an assigned queue number.

2. **Patient views medical history:**

   System Action: Loads the patient's personal medical timeline from the database. Each entry displays date, attending doctor, and diagnosis summary. Clicking an entry expands the full consultation detail.

3. **Patient pays invoice**:

   System Action: Redirects to the Billing section. Patient selects the outstanding invoice, chooses a payment method (cash, JazzCash, or Easypaisa), and confirms. Payment status is updated in real-time and a receipt is generated.

4. **Patient clicks notification bell**:

   System Action: Dropdown opens showing unread notifications (appointment reminders, billing alerts, doctor messages). All notifications in the dropdown are marked as read upon viewing.

5. **Patient edits profile**:

   System Action: Opens the profile edit form with pre-filled current information. Patient updates desired fields and clicks Save. The interface refreshes to reflect the updated profile data.

9.3.2 Doctor Dashboard

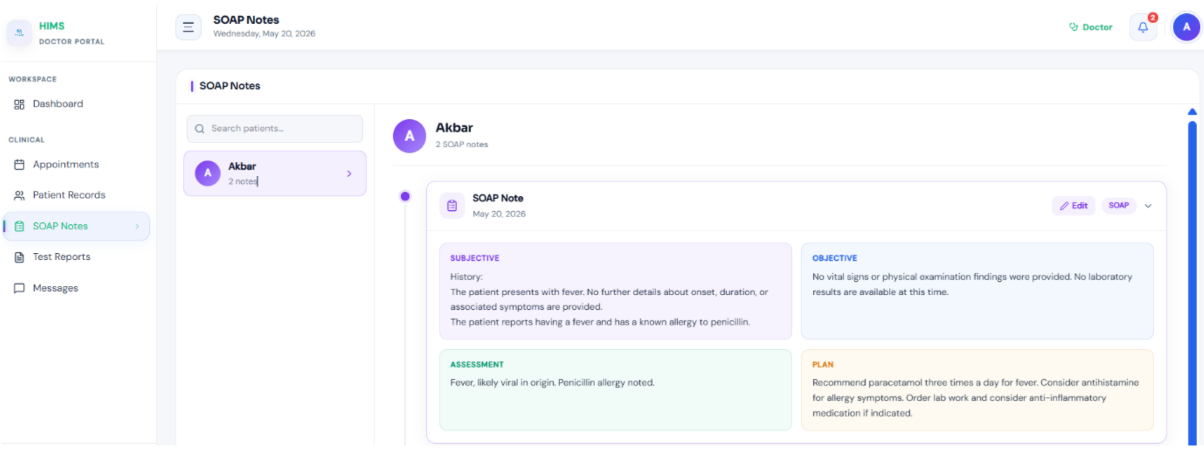


Figure 9.5 Doctor Dashboard

Purpose & Context:

The Doctor Dashboard is the primary workspace for doctors to manage their daily consultations, patient records, and AI-assisted documentation. It is designed to minimize administrative friction and keep the doctor focused on patient care. The dashboard provides a real-time view of the day's appointment queue, quick access to patient histories, and an integrated SOAP note editor with voice recording capabilities powered by the Vocacare AI engine.

Element Details:

Header

- Vocacare logo and welcome message with doctor's name
- Current time and date display
- Notification bell with alert dropdown
- User menu (Profile, Settings, Logout)

Tabs

- **Today's Appointments** — lists all scheduled patients for the current day with status indicators (Waiting, In Session, Completed, Missed)
- **My Patients** — full registry of doctor's patient list with search and filter capabilities
- **Profile** — doctor's personal and professional information management

Session Modal

- Left panel: Patient history summary (previous diagnoses, medications, visit dates)
- Center panel: SOAP notes editor (Subjective, Objective, Assessment, Plan) with auto-save
- Right panel: Voice recorder controls (Start, Stop, Playback)
- Action buttons: Submit Session, Finish Session

User Flow

1. **Doctor clicks "Start Session"**:

   System Action: Opens the fullscreen session modal. Loads the selected patient's medical history in the left panel and initializes the SOAP note template in the center. Requests microphone permissions if not already granted.

2. **Doctor edits SOAP Notes:**

   System Action: All changes to the SOAP note fields are auto-saved as a local draft. Unsaved or modified fields are visually highlighted to prevent accidental data loss.

3. **Doctor clicks "Start Recording":**

System Action: Requests microphone access from the browser. Upon permission, begins audio capture and sends the stream to the AI transcription engine. A pulsing "Recording…" indicator is displayed to signal active capture.

4. **Doctor clicks "Stop Recording"**:

   System Action: Finalizes the audio capture and saves the audio blob. The AI engine processes the recording, extracts medical entities (symptoms, diagnosis, medications), and populates the corresponding SOAP note fields for doctor review.

5. **Doctor clicks "Submit":**

   System Action: Validates all required SOAP fields. Uploads the completed notes and audio recording to the backend. Closes the session modal, marks the appointment status as "Completed," and displays a success toast: "Session Submitted."

6. **Doctor clicks "Finish Session"**:

   System Action: Closes the session modal without submitting if called prematurely, returning the doctor to the Appointments tab. If session was already submitted, it simply dismisses the modal.

7. **Doctor searches for a patient:**

   System Action: Triggers a live global search across the doctor's patient registry. Results appear in a dropdown as the doctor types. Clicking a result opens that patient's full profile page.

8. **Editing profile**:

   System Action: Allows the doctor to update personal and professional information (name, specialization, contact). Changes are saved via API and the UI refreshes to reflect the updates.

9.3.3 Receptionist Dashboard

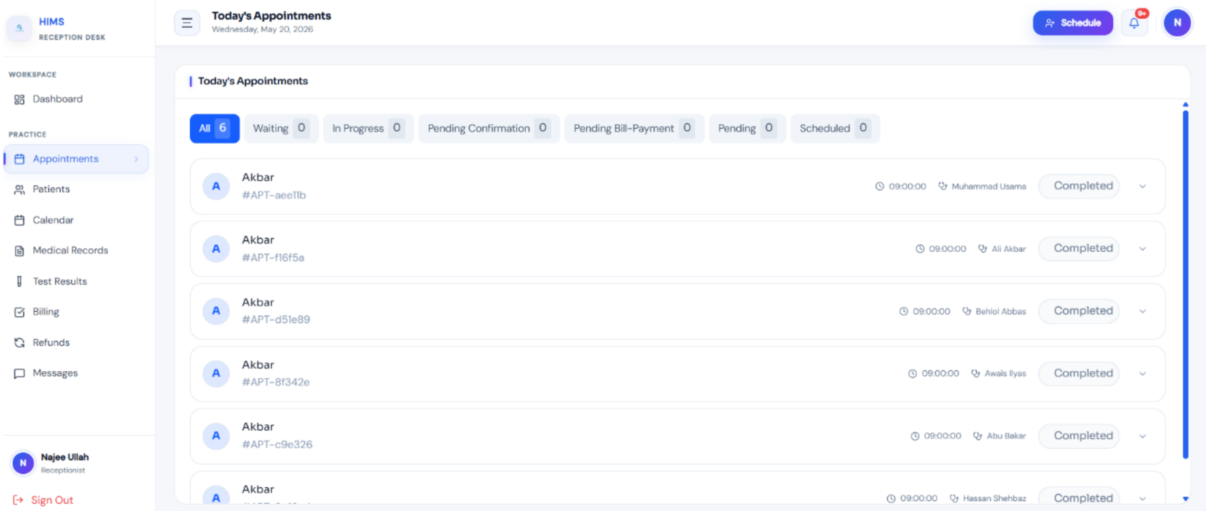


Figure 9.6 Receptionist Dashboard

Purpose & Context:

The Receptionist Dashboard is designed to support front-desk operations efficiently. Each receptionist is assigned to a specific doctor and can only view and manage operations related to that doctor's patient queue and appointments. The interface is kept simple and task-focused, enabling receptionists to handle patient check-ins, appointment bookings, registrations, and schedule monitoring without requiring deep technical knowledge.

Element Details:

Header

- Vocacare logo and welcome message
- Patient search bar for quick lookup
- Current time and date display
- Notification bell with dropdown
- User menu (Profile, Settings, Logout)

Main Modules

- **Today's Appointments** — table view of all scheduled patients for the assigned doctor, showing queue position, patient name, appointment time, and status
- **Patient Search** — search box with live results for looking up registered patients
- **Quick Actions** — shortcut buttons for Register New Patient and Book Appointment
- **Appointment Overview Table** — filterable list with check-in, reschedule, and cancel actions

User Flow

1. **Receptionist searches for a patient**:

   System Action: Queries the patient database in real-time as the receptionist types. Displays a list of matching results with patient name and ID. Clicking a result opens the full patient profile for the receptionist's review and action.

2. **Receptionist clicks "Check-In":**

   System Action: Marks the selected appointment as "Arrived" in the system. Updates the doctor's live appointment queue to reflect the patient's arrival.

3. **Click "Book Appointment":**

   System Action: Opens the appointment booking form. If a patient was previously selected, their information is pre-filled. The receptionist selects an available date slot from the doctor's schedule and confirms the booking.

4. **Clicking Notifications Icon:**

System Action: Dropdown opens displaying system alerts such as new appointment requests or queue updates. All viewed notifications are marked as read automatically.

5. **Clicking User Profile**:

   System Action: Opens a dropdown menu with options for Profile settings and Logout. Selecting Logout clears the active session and redirects to the Login page.

9.3.4 Admin Dashboard

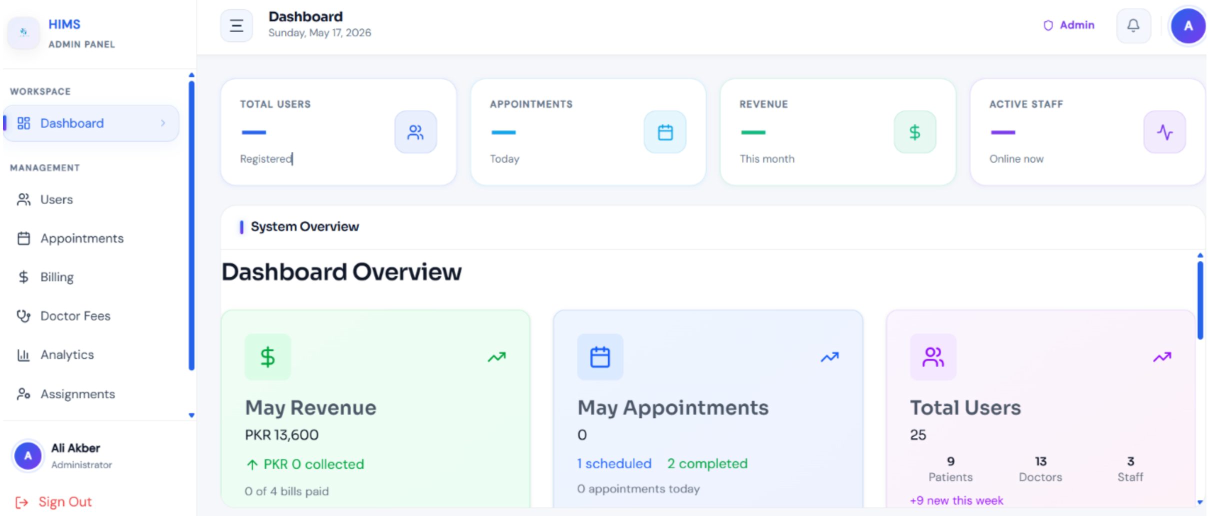


Figure 9.7 Admin Dashboard

Purpose & Context:

The Admin Dashboard provides hospital administrators with complete oversight and control of the Vocacare system. It serves as the management hub for user accounts, doctor-receptionist assignments, hospital analytics, billing monitoring, and system activity logs. The dashboard is data-rich and organized into clearly separated sections, enabling administrators to make informed operational decisions and maintain system integrity without requiring direct technical access to the backend.

Element Details:

Header

- Welcome message with admin name and role
- Global search bar in Users Tab(searches across users, departments, doctors, and logs)
- Current time and date display
- Notification bell with alert dropdown
- Admin menu (Profile, Logout)

Main Sections

- **Overview** — summary cards showing total patients, active doctors, today's appointments, and pending account approvals

- **Users** — full user management table with options to add, edit, activate, deactivate, or delete accounts; also handles patient account approval
- **Appointments** — system-wide appointment monitoring across all doctors with filter and export options
- **Billing** — overview of revenue, outstanding invoices, and payment history with export to PDF/CSV
- **Analytics** — charts and graphs for patient inflow trends, doctor workload, appointment frequency, and operational performance metrics

User Flow

1. **Admin uses global search:**

   System Action: Searches across all entities in the system via users tab — users, doctors, receptionists.

2. **Admin approves patient account**:

   System Action: Pending patient accounts appear in the Users section with a toggle action button which activates the account of the user.

3. **Admin views analytics:**

   System Action: Loads interactive charts showing appointment trends, revenue data.

4. **Admin edits system roles:**

   System Action: Opens the role and permissions editor. Admin can assign or revoke access to specific modules for each role type. Changes are saved and applied immediately across the system.

5. **Notifications dropdown:**

   System Action: Displays system-wide alerts including new account approval requests, failed system events, and billing anomalies. Events are marked as read upon being viewed.

6. **Logout**:

   System Action: Clears the admin session token from the cookie store and redirects to the Login page. All protected routes become inaccessible until re-authentication.

Chapter 10: Functionality & Performance Testing

10.1 Overview

This chapter documents the testing strategy, methodologies, test cases, and results conducted to validate the functionality and performance of the AI-Powered Hospital Information Management System (HIMS). The purpose of testing is to ensure that all system modules operate correctly, meet the specified functional requirements, and perform within acceptable response time thresholds under expected load conditions.

Testing was carried out across all major modules of the system including authentication, patient management, appointment scheduling, AI-assisted documentation, billing, and the administrative dashboard. Both manual functional testing and performance-based evaluation were conducted throughout the development lifecycle.

10.2 Testing Objectives

The primary objectives of the testing phase were:

- To verify that all functional requirements defined in Chapter 03 are correctly implemented and behave as expected.
- To validate that role-based access control correctly restricts and permits access based on user roles (Admin, Doctor, Receptionist, Patient).
- To confirm that the AI transcription and entity extraction module accurately processes and structures doctor-patient consultation audio.
- To ensure that all user-facing interfaces respond within acceptable time limits under normal operating conditions.
- To detect and document any defects, errors, or inconsistencies in system behavior and verify their resolution.

10.3 Testing Methodology

The following testing approaches were applied:

Manual Functional Testing: Each module was tested manually by team members acting in the role of each user type. Test cases were designed based on the functional requirements and use cases documented in Chapters 03 and 06. Testers followed defined test scenarios and recorded actual system behavior against expected behavior.

Black-Box Testing: Test cases were written based on system inputs and expected outputs without knowledge of the internal code structure. This approach was used primarily for UI-level testing of forms, navigation flows, and API responses.

Role-Based Access Testing: Each user role (Admin, Doctor, Receptionist, Patient) was tested in isolation to verify that the role-based access control (RBAC) system correctly grants and restricts access to specific modules, pages, and API endpoints.

Integration Testing: The interaction between frontend components and backend APIs was tested to ensure data flows correctly between modules — for example, verifying that a doctor's submitted SOAP notes are accurately stored and retrievable by the patient record module.

Performance Testing: Key system operations were timed to verify compliance with the performance requirements stated in Chapter 04. Response times were measured for page loads, data retrieval, appointment booking, and report generation under standard usage conditions.

10.4 Functional Test Cases and Results

10.4.1 Authentication Module

Test Case ID	Test Scenario	Expected Result	Actual Result	Status	Test Case ID
FT-01	User logs in with valid credentials	User redirected to role-specific dashboard	System redirected correctly	Pass	FT-01
FT-02	User enters invalid password	Error message displayed	Proper error displayed	Pass	FT-02
FT-03	Patient account pending approval attempts login	Access denied with approval message	System blocked login correctly	Pass	FT-03
FT-04	Password recovery request submitted	Password reset link generated	Email generated successfully	Pass	FT-04
FT-05	Session timeout after inactivity	User automatically logged out	Session expired correctly	Pass	FT-05

10.4.2 Patient Registration Testing

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
FT-06	Patient registers with valid information	Account created successfully	Account created	Pass
FT-07	User enters duplicate email	Registration blocked	Error displayed	Pass
FT-08	Password and confirm password mismatch	Registration prevented	Validation message displayed	Pass

FT-09	Admin approves patient account	Patient account activated	Activation successful	Pass
FT-10	Receptionist searches patient record	Correct patient displayed	Record retrieved successfully	Pass

10.4.3 Appointment & Queue Management Testing

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
FT-11	Patient books appointment	Appointment created with queue number	Appointment booked successfully	Pass
FT-12	Double booking attempt	Booking rejected	Conflict prevented	Pass
FT-13	Receptionist checks in patient	Queue status updated	Queue updated correctly	Pass
FT-14	Patient views queue position	Real-time queue displayed	Queue shown accurately	Pass
FT-15	Appointment cancellation	Appointment removed from schedule	Cancellation successful	Pass

10.4.4 AI Documentation Module Testing

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
FT-16	Doctor starts voice recording	Audio recording begins	Recording started correctly	Pass
FT-17	AI converts speech to text	Accurate transcription generated	Text generated successfully	Pass
FT-18	AI extracts medical entities	SOAP fields auto-filled	Extraction successful	Pass
FT-19	Doctor edits generated notes	Changes saved correctly	Notes updated successfully	Pass
FT-20	Doctor submits consultation	Consultation stored in database	Data saved successfully	Pass

10.4.5 Billing & Payment Testing

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
FT-21	Bill generated after consultation	Accurate invoice created	Invoice generated correctly	Pass
FT-22	Patient pays through JazzCash	Payment status updated	Payment completed	Pass

FT-23	Cancelled appointment refund	Refund processed	Refund generated successfully	Pass
FT-24	Admin views billing history	Billing records displayed	Data retrieved correctly	Pass

10.4.6 Role-Based Access Control Testing

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
FT-25	Doctor accesses admin dashboard	Access denied	Unauthorized access blocked	Pass
FT-26	Receptionist accesses assigned doctor appointments only	Limited data visible	Restriction enforced	Pass
FT-27	Patient attempts to access another patient's records	Access denied	Privacy maintained	Pass
FT-28	Admin modifies user permissions	Permissions updated	Changes applied successfully	Pass

10.5 Performance Testing

10.5.1 Performance Test Results

Operation	Expected Time	Actual Time	Status
User Login	Less than 5 sec	2.1 sec	Pass
Patient Record Retrieval	Less than 20 sec	4.8 sec	Pass
Appointment Booking	Less than 5 sec	2.9 sec	Pass
Billing Generation	Less than 7 sec	3.4 sec	Pass
Report Generation	Less than 7 sec	5.6 sec	Pass
Dashboard Loading	Less than 6 sec	3.2 sec	Pass
AI Transcription Processing	Less than 15 sec	9.7 sec	Pass

10.5.2 Concurrent User Testing

Number of Users	System Response	Status
25 Users	Stable performance	Pass
50 Users	No noticeable delay	Pass
75 Users	Minor response delay	Pass
100 Users	Acceptable performance maintained	Pass

10.6 Security Testing

Test Case ID	Security Scenario	Expected Result	Actual Result	Status
ST-01	Unauthorized page access	Access blocked	Blocked successfully	Pass
ST-02	SQL Injection attempt	Input sanitized	Attack prevented	Pass
ST-03	Session timeout after inactivity	User logged out automatically	Timeout successful	Pass
ST-04	Encrypted data transmission	SSL/TLS encryption active	Secure communication verified	Pass
ST-05	Invalid API request	Request denied	Unauthorized API blocked	Pass

10.7 Integration Testing

Test Case ID	Integration Scenario	Expected Result	Actual Result	Status
IT-01	Frontend sends login request to backend	Authentication successful	Working correctly	Pass
IT-02	SOAP notes saved to database	Notes retrievable from patient records	Successful	Pass
IT-03	Appointment booking updates queue	Queue updated automatically	Successful	Pass
IT-04	Billing module retrieves consultation charges	Accurate invoice generated	Successful	Pass
IT-05	Notification service sends alerts	Notifications delivered	Successful	Pass

10.8 Defects Identified and Resolved

Defect ID	Issue Description	Resolution
D-01	Queue status not refreshing automatically	Implemented real-time refresh logic
D-02	Duplicate appointment booking possible under rapid requests	Added backend slot validation
D-03	AI transcription delay during long recordings	Optimized audio processing pipeline
D-04	Session remained active after browser close	Improved session handling and token expiration
D-05	PDF report formatting issue	Updated PDF rendering templates

Chapter 11: References

1. *AI Scribe & Clinical Assist*. (n.d.). Retrieved from CureMD: https://www.curemd.com/ai-medical-scribe

2. Brian D Tran, Yunan Chen, Songzi Liu, Kai Zheng. (2020, May 01). *How does medical scribes' work inform development of speech-based clinical documentation technologies?* Retrieved from JAMIA: https://academic.oup.com/jamia/article-abstract/27/5/808/5809107

3. Cameron G. Shultz and Heather L. Holmstrom. (2015, May 28). *The Use of Medical Scribes in Health Care Settings: A Systematic Review and Future Directions.* Retrieved from JAFBFM: https://www.jabfm.org/content/28/3/371.short

4. Castillo, Sachin Shah, Deborah L. Burnet & Wei Wei Lee. (2021, July 11). *Impact of a medical scribe on clinical efficiency and quality in an academic general internal medicine practice.* Retrieved from Springer Nature Link: https://link.springer.com/article/10.1186/s12913-021-06710-y

5. Enrico Coiera, Baki Kocaballi, John Halamka & Liliana Laranjo . (2018, October 16). *The digital scribe.* Retrieved from npj digital medicine: https://www.nature.com/articles/s41746-018-0066-9

6. *Get time back . Move care forward.* (2025). Retrieved from Heidi: https://www.heidihealth.com/
7. Gregory Finley, Erik Edwards, Amanda Robinson, Michael Brenndoerfer, Najmeh Sadoughi, James Fone, Nico Axtmann, Mark Miller, David Suendermann-Oeft. (2018, June 01). *An automated medical scribe for documenting clinical encounters*. Retrieved from ACL Anthology: https://aclanthology.org/N18-5003/

8. jfgonsalves. (n.d.). *phlox*. Retrieved from Github: https://github.com/bloodworks-io/phlox

9. *Save 2 hours a day with an AI medical scribe.* (2025). Retrieved from NoteMD: https://www.notemd.ai/

10. *Sully AI*. (2025). Retrieved from Superhuman team of AI-employees for healthcare: https://www.sully.ai/

11. *The digital scribe. (2018, October 16). Retrieved from npj digital medicine : .* (n.d.). Retrieved from https://www.nature.com/articles/s41746-018-0066-9